

# 1. Presidio-SDV Anonymization Pipeline

```
# %% [markdown]
# # Demo: Presidio + SDV als Alternative zu anonym
#
# - Installation von Presidio & SDV (lokal, Windows 11)
# - Erzeugung eines fiktiven Textes mit PII
# - Erzeugung eines fiktiven tabellarischen Datensatzes
# - Deterministische & invertierbare Anonymisierung (Pseudonymisierung)
# - Nicht-invertierbare Anonymisierung (echte Anonymisierung)
#
# Ziel: Zeigen, dass Presidio (Text) + SDV (Tabellen) eine sinnvolle Alternative zu `anonym` sind.

# %% [markdown]
# ## 1. Installation (lokal, einmalig ausführen)
# Unter Windows 11 in einer venv/conda-Umgebung ausführen.

# %%
import sys
!{"sys.executable"} -m pip install presidio-analyzer presidio-anonymizer sdv pandas faker spacy
!{"sys.executable"} -m spacy download en_core_web_sm

# %% [markdown]
# ## 2. Presidio: Setup für Text-PII-Erkennung und -Anonymisierung

# %%
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine
from presidio_anonymizer.entities import OperatorConfig

# Analyzer & Anonymizer initialisieren
analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

# %% [markdown]
# ## 3. Fiktiver Text mit PII

# %%
text = (
    "My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt. "
    "My email address is max.mustermann@example.com, and my phone number is +1 415 555 1234."
)
print("Originaltext:\n", text)

# %% [markdown]
# ## 4. Deterministische & invertierbare Pseudonymisierung
#
# Idee:
# - Wir erzeugen deterministische Tokens (z.B. HASH) für jede erkannte Entität.
# - Diese Tokens können in einer Mapping-Tabelle gespeichert werden.
# - Damit ist die Pseudonymisierung **invertierbar**.

# %%
import hashlib

def deterministic_token(value: str, prefix: str) -> str:
    h = hashlib.sha256(value.encode("utf-8")).hexdigest()[:10]
    return f"{prefix}_{h}"

inverse_map = {}
```

```

def deterministic_replace(text: str):
    results = analyzer.analyze(text=text, language="en")
    anonymized_text = text
    for res in sorted(results, key=lambda r: r.start, reverse=True):
        original = text[res.start:res.end]
        token = deterministic_token(original, res.entity_type)
        inverse_map[token] = original
        anonymized_text = (
            anonymized_text[:res.start] + token + anonymized_text[res.end:]
        )
    return anonymized_text, results

det_text, det_results = deterministic_replace(text)
print("Deterministisch pseudonymisierter Text:\n", det_text)
print("\nInverse Map (Token -> Original):")
for k, v in inverse_map.items():
    print(k, "->", v)

# %% [markdown]
# ### 4.1 Invertierung (Rückführung der Pseudonyme)

# %%
def invert_text(pseudo_text: str, mapping: dict) -> str:
    inverted = pseudo_text
    for token, original in mapping.items():
        inverted = inverted.replace(token, original)
    return inverted

recovered_text = invert_text(det_text, inverse_map)
print("Rekonstruierter Originaltext:\n", recovered_text)

# %% [markdown]
# ## 5. Nicht-invertierbare Anonymisierung (neue Presidio-API)
#
# Wir nutzen OperatorConfig statt AnonymizerConfig.

# %%
operators = {
    "PERSON": OperatorConfig("replace", {"new_value": "[PERSON]"}),
    "PHONE_NUMBER": OperatorConfig("replace", {"new_value": "[PHONE]"}),
    "EMAIL_ADDRESS": OperatorConfig("replace", {"new_value": "[EMAIL]"}),
    "LOCATION": OperatorConfig("replace", {"new_value": "[LOCATION]"}),
}

analysis_results = analyzer.analyze(text=text, language="en")

anon_result = anonymizer.anonymize(
    text=text,
    analyzer_results=analysis_results,
    operators=operators
)

print("Nicht-invertierbar anonymisierter Text:\n", anon_result.text)

# %% [markdown]
# ## 6. SDV: Tabellarische Daten – synthetische & anonymisierte Variante

# %%
import pandas as pd
from faker import Faker

fake = Faker("de_DE")

data = []

```

```

for i in range(10):
    data.append(
        {
            "id": i + 1,
            "name": fake.name(),
            "email": fake.email(),
            "city": fake.city(),
            "age": fake.random_int(min=18, max=80),
        }
    )

df = pd.DataFrame(data)
print("Originale Tabelle:")
display(df)

# %% [markdown]
# ### 6.1 Deterministische & invertierbare Pseudonymisierung der Tabelle

# %%
name_map = {}
email_map = {}

def pseudo_value(value: str, prefix: str, mapping: dict) -> str:
    if value in mapping:
        return mapping[value]
    token = deterministic_token(value, prefix)
    mapping[value] = token
    return token

df_pseudo = df.copy()
df_pseudo["name_pseudo"] = df_pseudo["name"].apply(lambda v: pseudo_value(v, "NAME", name_map))
df_pseudo["email_pseudo"] = df_pseudo["email"].apply(lambda v: pseudo_value(v, "EMAIL", email_map))

print("Deterministisch pseudonymisierte Tabelle:")
display(df_pseudo)

# %% [markdown]
# ### 6.2 Invertierung der Pseudonymisierung

# %%
inv_name_map = {v: k for k, v in name_map.items()}
inv_email_map = {v: k for k, v in email_map.items()}

df_recovered = df_pseudo.copy()
df_recovered["name_recovered"] = df_recovered["name_pseudo"].apply(lambda v: inv_name_map.get(v, v))
df_recovered["email_recovered"] = df_recovered["email_pseudo"].apply(lambda v: inv_email_map.get(v, v))

print("Rekonstruierte Tabelle:")
display(df_recovered[["id", "name_recovered", "email_recovered", "city", "age"]])

# %% [markdown]
# ### 6.3 Nicht-invertierbare Anonymisierung via SDV (synthetische Daten)

# %%
from sdv.single_table import CTGANSynthesizer
from sdv.metadata import SingleTableMetadata

metadata = SingleTableMetadata()
metadata.detect_from_dataframe(df)

synthesizer = CTGANSynthesizer(metadata)
synthesizer.fit(df)

```

```

synthetic_df = synthesizer.sample(num_rows=10)

print("Synthetische Tabelle (nicht-invertierbar):")
display(synthetic_df)

# %% [markdown]
# ## 7. Fazit
#
# - **Presidio**:
#   - erkennt PII in Texten
#   - ermöglicht deterministische (invertierbare) und nicht-invertierbare Anonymisierung
# - **SDV**:
#   - erzeugt synthetische tabellarische Daten
#   - ideal für nicht-invertierbare Tabellenanonymisierung
#
# → Gemeinsam bilden Presidio + SDV eine moderne, sichere Alternative zu `anonym`.

```

```

Requirement already satisfied: presidio-analyzer in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (2.2.362)
Requirement already satisfied: presidio-anonymizer in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (2.2.362)
Requirement already satisfied: sdv in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (1.36.3)
Requirement already satisfied: pandas in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (2.3.3)
Requirement already satisfied: faker in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (24.14.1)
Requirement already satisfied: spacy in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (3.8.14)
Requirement already satisfied: phonenumbers<10.0.0,>=8.12 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from presidio-analyzer) (9.0.32)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from presidio-analyzer) (2.13.3)
Requirement already satisfied: pyyaml in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (from presidio-analyzer) (6.0.3)
Requirement already satisfied: regex in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (from presidio-analyzer) (2026.5.9)
Requirement already satisfied: tldextract in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-
packages (from presidio-analyzer) (5.3.1)
Requirement already satisfied: annotated-types>=0.6.0 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from pydantic<3.0.0,>=2.0.0->presidio-analyzer)
(0.7.0)
Requirement already satisfied: pydantic-core==2.46.3 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from pydantic<3.0.0,>=2.0.0->presidio-analyzer)
(2.46.3)
Requirement already satisfied: typing-extensions>=4.14.1 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from pydantic<3.0.0,>=2.0.0->presidio-analyzer)
(4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from pydantic<3.0.0,>=2.0.0->presidio-analyzer)
(0.4.2)
Requirement already satisfied: cryptography>=46.0.4 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from presidio-anonymizer) (46.0.7)
Requirement already satisfied: boto3<2.0.0,>=1.28 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (1.43.15)
Requirement already satisfied: botocore<2.0.0,>=1.31 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (1.43.15)
Requirement already satisfied: cloudpickle>=2.1.0 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (3.1.2)
Requirement already satisfied: graphviz>=0.13.2 in C:\Users\Nenad
Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (0.21)

```

Requirement already satisfied: numpy>=1.26.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (2.4.4)

Requirement already satisfied: tqdm>=4.29 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (4.67.3)

Requirement already satisfied: copulas>=0.12.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (0.14.1)

Requirement already satisfied: ctgan>=0.11.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (0.12.1)

Requirement already satisfied: deepecho>=0.7.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (0.8.1)

Requirement already satisfied: rdt>=1.18.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (1.21.0)

Requirement already satisfied: sdmetrics>=0.28.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (0.28.0)

Requirement already satisfied: platformdirs>=4.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from sdv) (4.9.6)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from pandas) (2026.1.post1)

Requirement already satisfied: tzdata>=2022.7 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from pandas) (2026.2)

Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from boto3<2.0.0,>=1.28->sdv) (1.1.0)

Requirement already satisfied: s3transfer<0.18.0,>=0.17.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from boto3<2.0.0,>=1.28->sdv) (0.17.1)

Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from botocore<2.0.0,>=1.31->sdv) (2.6.3)

Requirement already satisfied: six>=1.5 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (3.0.12)

Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (1.0.5)

Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (1.0.15)

Requirement already satisfied: cymem<2.1.0,>=2.0.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (2.0.13)

Requirement already satisfied: preshed<3.1.0,>=3.0.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (3.0.13)

Requirement already satisfied: thinc<8.4.0,>=8.3.12 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (8.3.13)

Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (1.1.3)

Requirement already satisfied: srsly<3.0.0,>=2.5.3 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (2.5.3)

Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (2.0.10)

Requirement already satisfied: weasel<2.0.0,>=1.0.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (1.0.0)

Requirement already satisfied: confection<2.0.0,>=1.3.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (1.3.3)

Requirement already satisfied: typer<1.0.0,>=0.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (0.24.2)

Requirement already satisfied: requests<3.0.0,>=2.13.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (2.33.1)

Requirement already satisfied: jinja2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (3.1.6)

Requirement already satisfied: setuptools in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (81.0.0)

Requirement already satisfied: packaging>=20.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from spacy) (26.1)

Requirement already satisfied: charset\_normalizer<4,>=2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from requests<3.0.0,>=2.13.0->spacy) (3.13)

Requirement already satisfied: certifi>=2023.5.7 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.4.22)

Requirement already satisfied: blis<1.4.0,>=1.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)

Requirement already satisfied: colorama in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from tqdm>=4.29->sdv) (0.4.6)

Requirement already satisfied: click>=8.2.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from typer<1.0.0,>=0.3.0->spacy) (8.3.3)

Requirement already satisfied: shellingham>=1.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)

Requirement already satisfied: rich>=12.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from typer<1.0.0,>=0.3.0->spacy) (15.0.0)

Requirement already satisfied: annotated-doc>=0.0.2 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from typer<1.0.0,>=0.3.0->spacy) (0.0.4)

Requirement already satisfied: cloudpathlib>=0.7.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.23.0)

Requirement already satisfied: smart-open>=5.2.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from weasel<2.0.0,>=1.0.0->spacy) (7.6.0)

Requirement already satisfied: httpx>=0.24.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from weasel<2.0.0,>=1.0.0->spacy) (0.28.1)

Requirement already satisfied: plotly>=5.10.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from copulas>=0.12.1->sdv) (6.7.0)

Requirement already satisfied: scipy>=1.12.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from copulas>=0.12.1->sdv) (1.17.1)

Requirement already satisfied: cffi>=2.0.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from cryptography>=46.0.4->presidio-anonymizer) (2.0.0)

Requirement already satisfied: pycparser in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from cffi>=2.0.0->cryptography>=46.0.4->presidio-anonymizer) (3.0)

Requirement already satisfied: torch>=2.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from ctgan>=0.11.1->sdv) (2.12.0)

Requirement already satisfied: anyio in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (4.13.0)

Requirement already satisfied: httpcore==1.\* in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (1.0.9)

Requirement already satisfied: h11>=0.16 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from httpcore==1.\*->httpx>=0.24.0->weasel<2.0.0,>=1.0.0->spacy) (0.16.0)

Requirement already satisfied: narwhals>=1.15.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from plotly>=5.10.0->copulas>=0.12.1->sdv) (2.21.2)

Requirement already satisfied: scikit-learn>=1.3.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from rdt>=1.18.2->sdv) (1.8.0)

Requirement already satisfied: markdown-it-py>=2.2.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (4.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (2.20.0)

Requirement already satisfied: mdurl~=0.1 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer<1.0.0,>=0.3.0->spacy) (0.1.2)

Requirement already satisfied: joblib>=1.3.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from scikit-learn>=1.3.1->rdt>=1.18.2->sdv) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.2.0 in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from scikit-learn>=1.3.1->rdt>=1.18.2->sdv) (3.6.0)

Requirement already satisfied: wrapt in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy) (2.1.2)

Requirement already satisfied: filelock in C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages (from torch>=2.3.0->ctgan>=0.11.1->sdv) (3.29.0)

Requirement already satisfied: sympy>=1.13.3 in C:\Users\Nenad

```
Balaneskovic\conda\envs\py312\Lib\site-packages (from torch>=2.3.0->ctgan>=0.11.1->sdv) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in C:\Users\Nenad
Balaneskovic\conda\envs\py312\Lib\site-packages (from torch>=2.3.0->ctgan>=0.11.1->sdv) (3.6.1)
Requirement already satisfied: fsspec>=0.8.5 in C:\Users\Nenad
Balaneskovic\conda\envs\py312\Lib\site-packages (from torch>=2.3.0->ctgan>=0.11.1->sdv) (2026.4.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in C:\Users\Nenad
Balaneskovic\conda\envs\py312\Lib\site-packages (from sympy>=1.13.3->torch>=2.3.0->ctgan>=0.11.1->sdv) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in C:\Users\Nenad
Balaneskovic\conda\envs\py312\Lib\site-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: requests-file>=1.4 in C:\Users\Nenad
Balaneskovic\conda\envs\py312\Lib\site-packages (from tldextract->presidio-analyzer) (3.0.1)
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
```

```
----- 0.0/12.8 MB ? eta -:--:--
----- 0.3/12.8 MB ? eta -:--:--
----- 0.3/12.8 MB ? eta -:--:--
----- 0.3/12.8 MB ? eta -:--:--
- ----- 0.5/12.8 MB 599.9 kB/s eta 0:00:21
-- ----- 0.8/12.8 MB 645.7 kB/s eta 0:00:19
-- ----- 0.8/12.8 MB 645.7 kB/s eta 0:00:19
-- ----- 0.8/12.8 MB 645.7 kB/s eta 0:00:19
----- 1.0/12.8 MB 592.2 kB/s eta 0:00:20
--- ----- 1.0/12.8 MB 592.2 kB/s eta 0:00:20
--- ----- 1.3/12.8 MB 610.3 kB/s eta 0:00:19
--- ----- 1.3/12.8 MB 610.3 kB/s eta 0:00:19
----- 1.6/12.8 MB 566.9 kB/s eta 0:00:20
----- 1.6/12.8 MB 566.9 kB/s eta 0:00:20
----- 1.6/12.8 MB 566.9 kB/s eta 0:00:20
----- 1.8/12.8 MB 565.6 kB/s eta 0:00:20
----- 1.8/12.8 MB 565.6 kB/s eta 0:00:20
----- 2.1/12.8 MB 541.3 kB/s eta 0:00:20
----- 2.1/12.8 MB 541.3 kB/s eta 0:00:20
----- 2.1/12.8 MB 541.3 kB/s eta 0:00:20
----- 2.1/12.8 MB 541.3 kB/s eta 0:00:20
----- 2.4/12.8 MB 506.6 kB/s eta 0:00:21
----- 2.4/12.8 MB 506.6 kB/s eta 0:00:21
----- 2.4/12.8 MB 506.6 kB/s eta 0:00:21
----- 2.6/12.8 MB 493.5 kB/s eta 0:00:21
----- 2.6/12.8 MB 493.5 kB/s eta 0:00:21
----- 2.9/12.8 MB 508.4 kB/s eta 0:00:20
----- 2.9/12.8 MB 508.4 kB/s eta 0:00:20
----- 3.1/12.8 MB 517.0 kB/s eta 0:00:19
----- 3.1/12.8 MB 517.0 kB/s eta 0:00:19
----- 3.1/12.8 MB 517.0 kB/s eta 0:00:19
----- 3.4/12.8 MB 508.4 kB/s eta 0:00:19
----- 3.4/12.8 MB 508.4 kB/s eta 0:00:19
----- 3.7/12.8 MB 506.1 kB/s eta 0:00:19
----- 3.7/12.8 MB 506.1 kB/s eta 0:00:19
----- 3.9/12.8 MB 512.9 kB/s eta 0:00:18
----- 3.9/12.8 MB 512.9 kB/s eta 0:00:18
----- 3.9/12.8 MB 512.9 kB/s eta 0:00:18
----- 4.2/12.8 MB 512.6 kB/s eta 0:00:17
----- 4.2/12.8 MB 512.6 kB/s eta 0:00:17
----- 4.2/12.8 MB 512.6 kB/s eta 0:00:17
----- 4.5/12.8 MB 508.4 kB/s eta 0:00:17
----- 4.7/12.8 MB 516.7 kB/s eta 0:00:16
----- 4.7/12.8 MB 516.7 kB/s eta 0:00:16
----- 5.0/12.8 MB 528.0 kB/s eta 0:00:15
----- 5.0/12.8 MB 528.0 kB/s eta 0:00:15
----- 5.0/12.8 MB 528.0 kB/s eta 0:00:15
----- 5.0/12.8 MB 528.0 kB/s eta 0:00:15
----- 5.2/12.8 MB 506.8 kB/s eta 0:00:15
```

```
----- 5.2/12.8 MB 506.8 kB/s eta 0:00:15
----- 5.2/12.8 MB 506.8 kB/s eta 0:00:15
----- 5.5/12.8 MB 502.3 kB/s eta 0:00:15
----- 5.5/12.8 MB 502.3 kB/s eta 0:00:15
----- 5.5/12.8 MB 502.3 kB/s eta 0:00:15
----- 5.8/12.8 MB 496.9 kB/s eta 0:00:15
----- 5.8/12.8 MB 496.9 kB/s eta 0:00:15
----- 6.0/12.8 MB 504.3 kB/s eta 0:00:14
----- 6.3/12.8 MB 509.1 kB/s eta 0:00:13
----- 6.3/12.8 MB 509.1 kB/s eta 0:00:13
----- 6.3/12.8 MB 509.1 kB/s eta 0:00:13
----- 6.6/12.8 MB 510.3 kB/s eta 0:00:13
----- 6.8/12.8 MB 515.9 kB/s eta 0:00:12
----- 6.8/12.8 MB 515.9 kB/s eta 0:00:12
----- 6.8/12.8 MB 515.9 kB/s eta 0:00:12
----- 7.1/12.8 MB 517.5 kB/s eta 0:00:12
----- 7.3/12.8 MB 522.5 kB/s eta 0:00:11
----- 7.3/12.8 MB 522.5 kB/s eta 0:00:11
----- 7.3/12.8 MB 522.5 kB/s eta 0:00:11
----- 7.6/12.8 MB 519.6 kB/s eta 0:00:11
----- 7.6/12.8 MB 519.6 kB/s eta 0:00:11
----- 7.6/12.8 MB 519.6 kB/s eta 0:00:11
----- 7.6/12.8 MB 519.6 kB/s eta 0:00:11
----- 7.9/12.8 MB 506.8 kB/s eta 0:00:10
----- 7.9/12.8 MB 506.8 kB/s eta 0:00:10
----- 7.9/12.8 MB 506.8 kB/s eta 0:00:10
----- 8.1/12.8 MB 506.4 kB/s eta 0:00:10
----- 8.1/12.8 MB 506.4 kB/s eta 0:00:10
----- 8.4/12.8 MB 509.9 kB/s eta 0:00:09
----- 8.4/12.8 MB 509.9 kB/s eta 0:00:09
----- 8.7/12.8 MB 510.8 kB/s eta 0:00:09
----- 8.7/12.8 MB 510.8 kB/s eta 0:00:09
----- 8.9/12.8 MB 511.2 kB/s eta 0:00:08
----- 8.9/12.8 MB 511.2 kB/s eta 0:00:08
----- 8.9/12.8 MB 511.2 kB/s eta 0:00:08
----- 9.2/12.8 MB 510.2 kB/s eta 0:00:08
----- 9.2/12.8 MB 510.2 kB/s eta 0:00:08
----- 9.4/12.8 MB 514.6 kB/s eta 0:00:07
----- 9.4/12.8 MB 514.6 kB/s eta 0:00:07
----- 9.7/12.8 MB 517.6 kB/s eta 0:00:07
----- 9.7/12.8 MB 517.6 kB/s eta 0:00:07
----- 9.7/12.8 MB 517.6 kB/s eta 0:00:07
----- 10.0/12.8 MB 513.0 kB/s eta 0:00:06
----- 10.0/12.8 MB 513.0 kB/s eta 0:00:06
----- 10.2/12.8 MB 513.7 kB/s eta 0:00:06
----- 10.2/12.8 MB 513.7 kB/s eta 0:00:06
----- 10.2/12.8 MB 513.7 kB/s eta 0:00:06
----- 10.5/12.8 MB 510.0 kB/s eta 0:00:05
----- 10.5/12.8 MB 510.0 kB/s eta 0:00:05
----- 10.5/12.8 MB 510.0 kB/s eta 0:00:05
----- 10.5/12.8 MB 510.0 kB/s eta 0:00:05
----- 10.7/12.8 MB 506.1 kB/s eta 0:00:05
----- 10.7/12.8 MB 506.1 kB/s eta 0:00:05
----- 11.0/12.8 MB 505.0 kB/s eta 0:00:04
----- 11.0/12.8 MB 505.0 kB/s eta 0:00:04
----- 11.0/12.8 MB 505.0 kB/s eta 0:00:04
----- 11.0/12.8 MB 505.0 kB/s eta 0:00:04
----- 11.3/12.8 MB 500.5 kB/s eta 0:00:04
----- 11.3/12.8 MB 500.5 kB/s eta 0:00:04
----- 11.3/12.8 MB 500.5 kB/s eta 0:00:04
----- 11.5/12.8 MB 498.6 kB/s eta 0:00:03
----- 11.5/12.8 MB 498.6 kB/s eta 0:00:03
----- 11.8/12.8 MB 497.8 kB/s eta 0:00:03
----- 11.8/12.8 MB 497.8 kB/s eta 0:00:03
```



```
----- -- 11.8/12.8 MB 497.8 kB/s eta 0:00:03
----- -- 12.1/12.8 MB 499.0 kB/s eta 0:00:02
----- -- 12.1/12.8 MB 499.0 kB/s eta 0:00:02
----- - 12.3/12.8 MB 497.9 kB/s eta 0:00:01
----- - 12.3/12.8 MB 497.9 kB/s eta 0:00:01
----- - 12.3/12.8 MB 497.9 kB/s eta 0:00:01
----- 12.6/12.8 MB 495.6 kB/s eta 0:00:01
----- 12.6/12.8 MB 495.6 kB/s eta 0:00:01
----- 12.6/12.8 MB 495.6 kB/s eta 0:00:01
----- 12.8/12.8 MB 493.4 kB/s 0:00:25

[38;5;2m[+] Download and installation successful[0m
You can now load the package via spacy.load('en_core_web_sm')
Originaltext:
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt. My email address is
max.mustermann@example.com, and my phone number is +1 415 555 1234.
Deterministisch pseudonymisierter Text:
My name is PERSON_ddd9b5b, and I live at LOCATION_69ce7653d7 in 12345 PERSON_9bba52b6be. My
email address is URL_7eb7c48f99ADDRESS_dd432348e66ee, and my phone number is
PHONE_NUMBER_8087d11524.

Inverse Map (Token -> Original):
PHONE_NUMBER_8087d11524 -> +1 415 555 1234
URL_a379a6f6ee -> example.com
EMAIL_ADDRESS_dd432348e6 -> max.mustermann@example.com
URL_7eb7c48f99 -> max.mu
PERSON_9bba52b6be -> Musterstadt
LOCATION_69ce7653d7 -> Musterstrasse 12
PERSON_ddd9b5b -> Max Mustermann
Rekonstruierter Originaltext:
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt. My email address is
max.muADDRESS_dd432348e66ee, and my phone number is +1 415 555 1234.
Nicht-invertierbar anonymisierter Text:
My name is [PERSON], and I live at [LOCATION] in 12345 [PERSON]. My email address is [EMAIL], and
my phone number is [PHONE].
Originale Tabelle:
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

</style>

|   | id | name                            | email                        | city          | age |
|---|----|---------------------------------|------------------------------|---------------|-----|
| 0 | 1  | Ahmed Otto                      | carolinroehrdanz@example.net | Celle         | 29  |
| 1 | 2  | Sabine Trüb                     | zhoefig@example.org          | Heinsberg     | 64  |
| 2 | 3  | Frau Reinhild Lübs MBA.         | pziegert@example.net         | Gransee       | 46  |
| 3 | 4  | Reinhild Naser                  | jsoelzer@example.com         | Emmendingen   | 45  |
| 4 | 5  | Milan Schüler                   | cbiggen@example.net          | Brilon        | 29  |
| 5 | 6  | Dr. Albina Jacobi Jäckel B.Eng. | urtehartung@example.net      | Aschaffenburg | 78  |
| 6 | 7  | Irmingard Kitzmann              | ywilmsen@example.net         | Husum         | 33  |

|   | id | name                     | email                        | city         | age |
|---|----|--------------------------|------------------------------|--------------|-----|
| 7 | 8  | Wilhelmine Döring        | rolf11@example.org           | Nauen        | 68  |
| 8 | 9  | Ing. German Adolph B.Sc. | hbuchholz@example.com        | Havelberg    | 57  |
| 9 | 10 | Reinhart Hermighausen    | schinkealejandro@example.org | Freudenstadt | 57  |

Deterministisch pseudonymisierte Tabelle:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }

```
.dataframe tbody tr th {
  vertical-align: top;
}

.dataframe thead th {
  text-align: right;
}
```

</style>

|   | id | name                                  | email                        | city          | age | name_pseudo     | email_pseudo     |
|---|----|---------------------------------------|------------------------------|---------------|-----|-----------------|------------------|
| 0 | 1  | Ahmed Otto                            | carolinroehrdanz@example.net | Celle         | 29  | NAME_999cac3df1 | EMAIL_b99b848bc6 |
| 1 | 2  | Sabine Trüb                           | zhoefig@example.org          | Heinsberg     | 64  | NAME_2a1f02858c | EMAIL_854c2cd392 |
| 2 | 3  | Frau Reinhild<br>Lübs MBA.            | pziegert@example.net         | Gransee       | 46  | NAME_1a5c6e0165 | EMAIL_aabb235454 |
| 3 | 4  | Reinhild Naser                        | jsoelzer@example.com         | Emmendingen   | 45  | NAME_e1c951abf1 | EMAIL_a24c1e99e5 |
| 4 | 5  | Milan Schüler                         | cbiggen@example.net          | Brilon        | 29  | NAME_a4ff33be49 | EMAIL_600bdb1d7e |
| 5 | 6  | Dr. Albina<br>Jacobi Jäckel<br>B.Eng. | urtehartung@example.net      | Aschaffenburg | 78  | NAME_c7dc680eea | EMAIL_0133680421 |
| 6 | 7  | Irmingard<br>Kitzmann                 | ywilmsen@example.net         | Husum         | 33  | NAME_8fb472fe81 | EMAIL_67b54db7f3 |
| 7 | 8  | Wilhelmine<br>Döring                  | rolf11@example.org           | Nauen         | 68  | NAME_1b88815181 | EMAIL_6c39cfc2a1 |
| 8 | 9  | Ing. German<br>Adolph B.Sc.           | hbuchholz@example.com        | Havelberg     | 57  | NAME_9ceb0f590c | EMAIL_eff5796e97 |
| 9 | 10 | Reinhart<br>Hermighausen              | schinkealejandro@example.org | Freudenstadt  | 57  | NAME_34b9354d70 | EMAIL_d263f6a98f |

Rekonstruierte Tabelle:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }

```
.dataframe tbody tr th {
  vertical-align: top;
}
```

```
.dataframe thead th {
    text-align: right;
}
```

</style>

|   | id | name_recovered                  | email_recovered              | city          | age |
|---|----|---------------------------------|------------------------------|---------------|-----|
| 0 | 1  | Ahmed Otto                      | carolinroehrdanz@example.net | Celle         | 29  |
| 1 | 2  | Sabine Trüb                     | zhoefig@example.org          | Heinsberg     | 64  |
| 2 | 3  | Frau Reinhild Lübs MBA.         | pziegert@example.net         | Gransee       | 46  |
| 3 | 4  | Reinhild Naser                  | jsoelzer@example.com         | Emmendingen   | 45  |
| 4 | 5  | Milan Schüler                   | cbiggen@example.net          | Brilon        | 29  |
| 5 | 6  | Dr. Albina Jacobi Jäckel B.Eng. | urtehartung@example.net      | Aschaffenburg | 78  |
| 6 | 7  | Irmingard Kitzmann              | ywilmsen@example.net         | Husum         | 33  |
| 7 | 8  | Wilhelmine Döring               | rolf11@example.org           | Nauen         | 68  |
| 8 | 9  | Ing. German Adolph B.Sc.        | hbuchholz@example.com        | Havelberg     | 57  |
| 9 | 10 | Reinhart Hermighausen           | schinkealejandro@example.org | Freudenstadt  | 57  |

```
C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages\sdv\single_table\base.py:178:
FutureWarning: The 'SingleTableMetadata' is deprecated. Please use the new 'Metadata' class for
synthesizers.
    warnings.warn(DEPRECATION_MSG, FutureWarning)
C:\Users\Nenad Balaneskovic\.conda\envs\py312\Lib\site-packages\sdv\single_table\base.py:134:
UserWarning: We strongly recommend saving the metadata using 'save_to_json' for replicability in
future SDV versions.
    warnings.warn(

Synthetische Tabelle (nicht-invertierbar):
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }

```
.dataframe tbody tr th {
    vertical-align: top;
}

.dataframe thead th {
    text-align: right;
}
```

</style>

|   | id       | name                            | email                     | city               | age |
|---|----------|---------------------------------|---------------------------|--------------------|-----|
| 0 | 5451002  | Ahmed Otto                      | hayesjorge@example.net    | Timothytown        | 78  |
| 1 | 6046429  | Irmingard Kitzmann              | yadams@example.net        | Duanehaven         | 29  |
| 2 | 11509909 | Dr. Albina Jacobi Jäckel B.Eng. | taylor17@example.net      | Summersborough     | 53  |
| 3 | 7602670  | Ahmed Otto                      | anthony58@example.com     | West Jenniferhaven | 29  |
| 4 | 5066721  | Wilhelmine Döring               | garciaheather@example.com | Guerraland         | 52  |

|   | id       | name                            | email                     | city          | age |
|---|----------|---------------------------------|---------------------------|---------------|-----|
| 5 | 316698   | Frau Reinhild Lübs MBA.         | bcampbell@example.com     | South Carrie  | 42  |
| 6 | 7651665  | Ing. German Adolph B.Sc.        | djones@example.net        | New Timmouth  | 29  |
| 7 | 4488593  | Reinhart Hermighausen           | smithjennifer@example.com | Bridgeshaven  | 48  |
| 8 | 1490125  | Reinhart Hermighausen           | desiree85@example.net     | Burtonchester | 46  |
| 9 | 12336541 | Dr. Albina Jacobi Jäckel B.Eng. | tara64@example.org        | East Tara     | 46  |

## 2. DAG-Version

```
# dag_presidio_sdv_smoketest.py

from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.python import PythonOperator

def presidio_sdv_smoketest(**context):
    # --- 1. Presidio: Setup für Text-PII-Erkennung und -Anonymisierung ---
    from presidio_analyzer import AnalyzerEngine
    from presidio_anonymizer import AnonymizerEngine
    from presidio_anonymizer.entities import OperatorConfig

    analyzer = AnalyzerEngine()
    anonymizer = AnonymizerEngine()

    # --- 2. Fiktiver Text mit PII ---
    text = (
        "My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt. "
        "My email address is max.mustermann@example.com, and my phone number is +1 415 555 1234."
    )
    print("Originaltext:\n", text)

    # --- 3. Deterministische & invertierbare Pseudonymisierung (Text) ---
    import hashlib

    def deterministic_token(value: str, prefix: str) -> str:
        h = hashlib.sha256(value.encode("utf-8")).hexdigest()[:10]
        return f"{prefix}_{h}"

    inverse_map = {}

    def deterministic_replace(text_: str):
        results = analyzer.analyze(text=text_, language="en")
        anonymized_text = text_
        for res in sorted(results, key=lambda r: r.start, reverse=True):
            original = text_[res.start:res.end]
            token = deterministic_token(original, res.entity_type)
            inverse_map[token] = original
            anonymized_text = (
                anonymized_text[:res.start] + token + anonymized_text[res.end:]
            )
        return anonymized_text, results

    det_text, det_results = deterministic_replace(text)
    print("Deterministisch pseudonymisierter Text:\n", det_text)
    print("\nInverse Map (Token -> Original):")
```

```

for k, v in inverse_map.items():
    print(k, "->", v)

def invert_text(pseudo_text: str, mapping: dict) -> str:
    inverted = pseudo_text
    for token, original in mapping.items():
        inverted = inverted.replace(token, original)
    return inverted

recovered_text = invert_text(det_text, inverse_map)
print("Rekonstruierter Originaltext:\n", recovered_text)

# --- 4. Nicht-invertierbare Anonymisierung (Text) ---
operators = {
    "PERSON": OperatorConfig("replace", {"new_value": "[PERSON]"}),
    "PHONE_NUMBER": OperatorConfig("replace", {"new_value": "[PHONE]"}),
    "EMAIL_ADDRESS": OperatorConfig("replace", {"new_value": "[EMAIL]"}),
    "LOCATION": OperatorConfig("replace", {"new_value": "[LOCATION]"}),
}

analysis_results = analyzer.analyze(text=text, language="en")

anon_result = anonymizer.anonymize(
    text=text,
    analyzer_results=analysis_results,
    operators=operators,
)

print("Nicht-invertierbar anonymisierter Text:\n", anon_result.text)

# --- 5. SDV: Tabellarische Daten - synthetische & anonymisierte Variante ---
import pandas as pd
from faker import Faker

fake = Faker("de_DE")

data = []
for i in range(10):
    data.append(
        {
            "id": i + 1,
            "name": fake.name(),
            "email": fake.email(),
            "city": fake.city(),
            "age": fake.random_int(min=18, max=80),
        }
    )

df = pd.DataFrame(data)
print("Originale Tabelle:")
print(df.to_string(index=False))

# --- 5.1 Deterministische & invertierbare Pseudonymisierung der Tabelle ---
name_map = {}
email_map = {}

def pseudo_value(value: str, prefix: str, mapping: dict) -> str:
    if value in mapping:
        return mapping[value]
    token = deterministic_token(value, prefix)
    mapping[value] = token
    return token

df_pseudo = df.copy()

```

```

df_pseudo["name_pseudo"] = df_pseudo["name"].apply(
    lambda v: pseudo_value(v, "NAME", name_map)
)
df_pseudo["email_pseudo"] = df_pseudo["email"].apply(
    lambda v: pseudo_value(v, "EMAIL", email_map)
)

print("Deterministisch pseudonymisierte Tabelle:")
print(df_pseudo.to_string(index=False))

# --- 5.2 Invertierung der Pseudonymisierung ---
inv_name_map = {v: k for k, v in name_map.items()}
inv_email_map = {v: k for k, v in email_map.items()}

df_recovered = df_pseudo.copy()
df_recovered["name_recovered"] = df_recovered["name_pseudo"].apply(
    lambda v: inv_name_map.get(v, v)
)
df_recovered["email_recovered"] = df_recovered["email_pseudo"].apply(
    lambda v: inv_email_map.get(v, v)
)

print("Rekonstruierte Tabelle:")
print(
    df_recovered[["id", "name_recovered", "email_recovered", "city", "age"]].to_string(
        index=False
    )
)

# --- 5.3 Nicht-invertierbare Anonymisierung via SDV (synthetische Daten) ---
from sdv.single_table import CTGANSynthesizer
from sdv.metadata import SingleTableMetadata

metadata = SingleTableMetadata()
metadata.detect_from_dataframe(df)

synthesizer = CTGANSynthesizer(metadata)
synthesizer.fit(df)

synthetic_df = synthesizer.sample(num_rows=10)

print("Synthetische Tabelle (nicht-invertierbar):")
print(synthetic_df.to_string(index=False))

# Smoketest-Fazit im Log
print(
    "\nSmoketest Presidio + SDV erfolgreich: "
    "Text-PII erkannt, pseudonymisiert, invertiert; "
    "nicht-invertierbar anonymisiert; "
    "Tabellen pseudonymisiert, invertiert und synthetisch generiert."
)

default_args = {
    "owner": "data-privacy",
    "depends_on_past": False,
    "retries": 0,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    dag_id="presidio_sdv_smoketest",
    default_args=default_args,
    description="Smoketest-DAG für Presidio + SDV als Alternative zu anonym",

```

```
schedule_interval=None,
start_date=datetime(2024, 1, 1),
catchup=False,
tags=["privacy", "presidio", "sdv", "smoketest"],
) as dag:

    run_presidio_sdv_smoketest = PythonOperator(
        task_id="run_presidio_sdv_smoketest",
        python_callable=presidio_sdv_smoketest,
    )

    run_presidio_sdv_smoketest
```

---

## 3. EXPLANATION SECTION

---

### Chapter 1/15 — Introduction & Conceptual Foundations

---

This first chapter lays the conceptual groundwork for the entire 15-part series.

It explains *what the notebook is, why it exists, what problems it solves, and how the different components fit together.*

The goal is to offer the user a deep, intuitive understanding before we dive into the technical details.

---

## 1. Purpose of the Notebook

---

The notebook demonstrates a complete, end-to-end workflow for **detecting**, **transforming**, and **protecting** sensitive information in both:

- **unstructured text** (sentences, documents, messages)
- **structured tabular data** (tables, CSV files, databases)

It uses two modern open-source libraries:

### Presidio

A Microsoft-developed framework for:

- detecting Personally Identifiable Information (PII) in text
- anonymizing or pseudonymizing that information

### SDV (Synthetic Data Vault)

A library for:

- generating synthetic tabular data
- preserving statistical properties
- ensuring that the synthetic data cannot be traced back to individuals

Together, these tools form a powerful alternative to older anonymization libraries.

---

## 2. What the Notebook Demonstrates

---

The notebook walks through five major capabilities:

### 1. PII Detection in Text

Using Presidio's NLP-based recognizers to identify:

- names
- locations
- email addresses
- phone numbers
- other sensitive entities

## 2. Deterministic Pseudonymization (Text)

Replacing each detected entity with a **stable, reversible token**, such as:

```
PERSON_dddfab9b5b  
EMAIL_123abc4567
```

This allows:

- reversible transformations
- consistent pseudonyms across documents
- safe storage of mapping tables

## 3. Non-invertible Anonymization (Text)

Replacing entities with **irreversible placeholders**, such as:

```
[ PERSON ]  
[ EMAIL ]  
[ PHONE ]
```

This ensures:

- no reconstruction is possible
- no mapping table exists
- maximum privacy

## 4. Deterministic Pseudonymization (Tables)

Applying the same reversible hashing logic to:

- names
- email addresses

in a structured dataset.

## 5. Synthetic Data Generation (Tables)

Using SDV's CTGAN model to generate **entirely new rows** that:

- resemble the original data statistically
- contain no real individuals
- cannot be reverse-engineered

---

# 3. Why These Techniques Matter

---

Modern data workflows often require:



- sharing data with external teams
- training machine learning models
- debugging pipelines
- creating demos or prototypes
- performing analytics without exposing real individuals

However, raw data often contains sensitive information:

- names
- addresses
- phone numbers
- emails
- IDs
- demographic attributes

This creates a tension:

**How can we use data without exposing private information?**

The notebook demonstrates two complementary strategies:

---

## A. Pseudonymization (Reversible)

Useful when:

- we need to restore original values later
- we need consistent identifiers across datasets
- we want to track entities without revealing them

Example use cases:

- linking records across systems
  - debugging data pipelines
  - performing longitudinal analysis
- 

## B. Anonymization (Irreversible)

Useful when:

- we must guarantee privacy
- we want to share data externally
- we want to publish datasets
- we want to eliminate re-identification risk

Example use cases:

- public datasets
  - research
  - demos
  - training ML models
- 

# 4. Why Presidio and SDV Work Well Together

---

Presidio and SDV solve **different** but **complementary** problems.

### Presidio

Focuses on **textual PII**:

- names
- emails
- phone numbers
- addresses
- locations

It is ideal for:

- documents
- logs
- messages
- free-form text

## SDV

Focuses on **tabular data**:

- rows and columns
- relational structures
- numerical distributions
- categorical patterns

It is ideal for:

- databases
- CSV files
- analytics tables

Together, they cover **both major data types** used in real systems.

---

## 5. The Example Data Used in the Notebook

---

To demonstrate the tools, the notebook uses:

### A. A synthetic English sentence containing PII

```
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt.  
My email address is max.mustermann@example.com, and my phone number is +1 415 555 1234.
```

This sentence contains:

- PERSON
- LOCATION
- EMAIL\_ADDRESS
- PHONE\_NUMBER

Presidio detects all of them.

---

### B. A synthetic table generated with Faker

The table contains:

- id
- name
- email
- city

- age

All values are **fake**, but realistic.

This allows:

- safe demonstration
- no privacy concerns
- reproducible examples

---

## 6. The Two Core Concepts: Pseudonymization vs. Anonymization

---

Before diving into the code, it's important to understand the difference.

---

### Pseudonymization (Reversible)

#### Definition

Replacing sensitive data with **consistent, reversible tokens**.

#### Example

```
Max Mustermann → PERSON_dddfab9b5b
```

#### Properties

- deterministic
- reversible
- consistent across datasets
- useful for linking records

#### Risk

If the mapping table is leaked, the original data can be reconstructed.

---

### Anonymization (Irreversible)

#### Definition

Replacing sensitive data with **non-reversible placeholders** or generating **synthetic data**.

#### Example

```
Max Mustermann → [PERSON]
```

#### Properties

- irreversible
- no mapping table
- safe for external sharing
- eliminates re-identification risk

## Risk

None, if done correctly.

---

## 7. What We Will Learn in the Next Chapters

---

This 15-part series will teach us:

- how Presidio detects PII
- how deterministic hashing works
- how to build reversible pseudonymization pipelines
- how to build irreversible anonymization pipelines
- how SDV generates synthetic data
- how to interpret the outputs
- how to combine these techniques effectively

By the end, we will understand:

- the full architecture
  - the reasoning behind each step
  - the strengths and limitations
  - how to adapt the code to our own use cases
- 

## 8. Summary of Chapter 1

---

This chapter introduced:

- the purpose of the notebook
- the tools used (Presidio + SDV)
- the difference between pseudonymization and anonymization
- the structure of the demonstration
- the goals of the 15-chapter series

We now have the conceptual foundation needed to understand the technical details that follow.

---

## Chapter 2/15 — Installation & Environment Setup

---

This chapter explains the installation section of the notebook in depth.

We will explore:

- why each package is installed
- how they interact
- what role they play in the anonymization pipeline
- how the environment is prepared for Presidio and SDV
- what the installation output means

This chapter sets the foundation for understanding the rest of the notebook.

---

## 1. Overview of the Installation Step

---

The notebook begins with:

```
import sys
!"{sys.executable}" -m pip install presidio-analyzer presidio-anonymizer sdv pandas faker spacy
!"{sys.executable}" -m spacy download en_core_web_sm
```

This block performs two essential tasks:

1. **Install all required Python packages**
2. **Download the English spaCy model** used by Presidio

The installation is executed using:

```
"{sys.executable}" -m pip install ...
```

This ensures that the packages are installed **into the exact Python environment** that Jupyter Notebook is running in. This avoids the common issue where:

- Jupyter uses one Python interpreter
- `pip install` installs into another

Using `sys.executable` guarantees consistency.

---

## 2. Why Each Package Is Installed

---

Let's break down each installed package and its purpose.

---

### 2.1 presidio-analyzer

Presidio Analyzer is responsible for:

- detecting PII in text
- identifying entity types (PERSON, EMAIL\_ADDRESS, PHONE\_NUMBER, LOCATION, etc.)
- using NLP models and pattern recognizers

It is the **core engine** that scans text and returns structured information about sensitive entities.

What it provides:

- `AnalyzerEngine`
- built-in recognizers
- NLP pipeline integration
- scoring and confidence thresholds

Without this package, no PII detection would be possible.

---

### 2.2 presidio-anonymizer

Presidio Anonymizer performs the **transformation** of detected PII.

It supports:

- replacement
- masking
- redaction
- hashing

- encryption (optional)
- custom operators

In this notebook, we use:

- **OperatorConfig("replace")** for non-invertible anonymization
- **custom deterministic hashing** for invertible pseudonymization

Presidio Analyzer finds the PII.

Presidio Anonymizer transforms it.

---

## 2.3 sdv

SDV (Synthetic Data Vault) is a framework for generating **synthetic tabular data**.

It includes:

- CTGAN (a GAN-based model for tabular data)
- GaussianCopula
- CopulaGAN
- Metadata management

In this notebook, SDV is used to:

- learn the statistical structure of the input table
- generate new rows that resemble the original
- ensure the synthetic data is **non-invertible**

This is essential for demonstrating anonymization of structured data.

---

## 2.4 pandas

Pandas is the standard Python library for:

- dataframes
- tabular manipulation
- CSV-like structures

It is used to:

- create the example table
- apply pseudonymization
- display results
- pass data into SDV

Without pandas, the table-based part of the notebook would not be possible.

---

## 2.5 faker

Faker is used to generate **synthetic but realistic** personal data:

- names
- emails
- cities
- addresses

This ensures:

- no real personal data is used
- the demonstration is safe

- the examples are reproducible

The notebook uses the German locale:

```
fake = Faker("de_DE")
```

This produces realistic German names and cities.

---

## 2.6 spacy

spaCy is a modern NLP library used by Presidio for:

- tokenization
- sentence segmentation
- part-of-speech tagging
- named entity recognition (NER)

Presidio Analyzer relies on spaCy to process text before applying recognizers.

---

## 2.7 en\_core\_web\_sm

This is the **English spaCy model**.

Presidio requires a language model to:

- tokenize text
- identify sentence boundaries
- provide linguistic features

The notebook uses:

```
!("{sys.executable}" -m spacy download en_core_web_sm
```

This installs the small English model, which is:

- lightweight
- fast
- sufficient for PII detection

Presidio does not ship with its own NLP model; it depends on spaCy.

---

## 3. Why the English Model Is Used

---

Even though the example text contains German names and addresses, the sentence is written in **English**, and Presidio's built-in recognizers are optimized for English.

Presidio does **not** include German recognizers by default.

Using the English model ensures:

- PERSON entities are detected
- EMAIL\_ADDRESS is detected
- PHONE\_NUMBER is detected
- LOCATION is detected (based on patterns)

If a German model were used, Presidio would still not have German recognizers unless manually added.

Thus, the English model is the correct choice.

---

## 4. What Happens During Installation

---

When we run the installation cell, several things occur:

✓ spaCy downloads the model

We see messages like:

```
You can now load the package via spacy.load('en_core_web_sm')
```

This confirms the model is installed.

✓ Presidio installs its components

We may see:

- dependency resolution
- installation of recognizers
- installation of anonymizer operators

✓ SDV installs machine learning dependencies

This includes:

- PyTorch (if needed)
- numpy
- scipy
- sklearn

✓ Faker installs locale data

This includes:

- name dictionaries
- city lists
- email patterns

---

## 5. Verifying the Installation

---

After installation, the notebook imports:

```
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine
from presidio_anonymizer.entities import OperatorConfig
```

If these imports succeed:

- Presidio is installed correctly
- spaCy is installed correctly
- the English model is available



If any import fails, it usually means:

- Jupyter is using a different Python interpreter
- the model was installed into a different environment

Using `sys.executable` avoids this problem.

---

## 6. Why This Installation Step Matters

---

This step ensures that:

- the environment is consistent
- all required libraries are available
- Presidio can run NLP pipelines
- SDV can train generative models
- Faker can produce synthetic data
- pandas can manipulate tables

Without this foundation, none of the later steps would work.

---

## 7. Summary of Chapter 2

---

In this chapter, we explored:

- the purpose of each installed package
- how Presidio depends on spaCy
- why the English model is used
- how SDV fits into the workflow
- how the installation ensures environment consistency
- what the installation output means

We now understand the technical foundation required for the rest of the notebook.

---

## Chapter 3/15 — Understanding Presidio's Architecture: Analyzer & Anonymizer

---

This chapter explains the internal architecture of **Presidio**, focusing on the two core components used in the notebook:

- the **AnalyzerEngine**
- the **AnonymizerEngine**

Understanding these two components is essential because they form the backbone of all PII detection and transformation operations in the notebook.

This chapter will cover:

1. What Presidio is designed to do
  2. How the AnalyzerEngine works internally
  3. How the AnonymizerEngine works internally
  4. How spaCy integrates into Presidio
  5. How recognizers detect PII
  6. How operators transform PII
  7. How these components interact in the notebook
-

# 1. What Presidio Is Designed to Do

---

Presidio is an open-source framework created to:

- **detect** sensitive information in text
- **classify** that information into entity types
- **transform** or **remove** that information

It is built around two major ideas:

## A. PII Detection (Analyzer)

Identify sensitive entities such as:

- PERSON
- EMAIL\_ADDRESS
- PHONE\_NUMBER
- LOCATION
- CREDIT\_CARD
- IP\_ADDRESS
- DATE\_TIME
- and many more

## B. PII Transformation (Anonymizer)

Apply transformations such as:

- replacement
- masking
- hashing
- encryption
- redaction

Presidio is modular, meaning:

- the analyzer and anonymizer are separate
- we can use one without the other
- we can plug in custom logic at any point

This modularity is what makes Presidio powerful and flexible.

---

## 2. The AnalyzerEngine: How Presidio Detects PII

---

The notebook initializes the analyzer with:

```
analyzer = AnalyzerEngine()
```

This creates an instance of the **AnalyzerEngine**, which is responsible for:

- loading recognizers
- running NLP preprocessing
- applying pattern-based and ML-based detection
- returning structured PII results

Let's break down how it works internally.

---

## 2.1 The Analyzer Pipeline

When we call:

```
results = analyzer.analyze(text=text, language="en")
```

Presidio performs the following steps:

### Step 1 — NLP Preprocessing

Presidio uses spaCy to:

- tokenize the text
- split it into sentences
- identify part-of-speech tags
- extract named entities
- normalize whitespace and punctuation

This produces an **NLP artifacts object**, which is passed to recognizers.

### Step 2 — Recognizer Selection

Presidio has a registry of recognizers.

Each recognizer is responsible for detecting a specific type of PII.

Examples:

- **EmailRecognizer** → EMAIL\_ADDRESS
- **PhoneRecognizer** → PHONE\_NUMBER
- **SpacyRecognizer** → PERSON, LOCATION, ORG
- **RegexRecognizer** → IP\_ADDRESS, CREDIT\_CARD

When we specify:

```
language="en"
```

Presidio loads all recognizers that support English.

### Step 3 — Entity Detection

Each recognizer scans the text and returns:

- entity type
- start index
- end index
- confidence score

For example:

```
PERSON: Max Mustermann (start=11, end=25)
EMAIL_ADDRESS: max.mustermann@example.com
PHONE_NUMBER: +1 415 555 1234
LOCATION: Musterstrasse 12
```

### Step 4 — Aggregation & Conflict Resolution

If multiple recognizers detect overlapping entities:

- Presidio chooses the one with the highest confidence
- or merges them if appropriate

### Step 5 — Final Result List

The analyzer returns a list of **RecognizerResult** objects.

Each contains:

- entity\_type
- start
- end
- score

These results are then used by the anonymizer or by custom logic.

---

## 3. The AnonymizerEngine: How Presidio Transforms PII

---

The notebook initializes the anonymizer with:

```
anonymizer = AnonymizerEngine()
```

The anonymizer takes:

- the original text
- the list of detected entities
- a set of transformation rules

and produces a new text where PII has been transformed.

---

### 3.1 How the AnonymizerEngine Works

When we call:

```
anon_result = anonymizer.anonymize(  
    text=text,  
    analyzer_results=analysis_results,  
    operators=operators  
)
```

Presidio performs:

#### Step 1 — Sort Entities by Position

Entities are sorted from **end to start** to avoid index shifting.

#### Step 2 — Apply Operators

Each entity type is mapped to an operator.

Example from the notebook:

```
operators = {  
    "PERSON": OperatorConfig("replace", {"new_value": "[PERSON]"}),  
    "PHONE_NUMBER": OperatorConfig("replace", {"new_value": "[PHONE]"}),  
    "EMAIL_ADDRESS": OperatorConfig("replace", {"new_value": "[EMAIL]"}),  
    "LOCATION": OperatorConfig("replace", {"new_value": "[LOCATION]"}),  
}
```

This means:

- PERSON → replace with [PERSON]
- PHONE\_NUMBER → replace with [PHONE]
- EMAIL\_ADDRESS → replace with [EMAIL]
- LOCATION → replace with [LOCATION]

### Step 3 — Construct the Output Text

The anonymizer builds a new string by:

- copying non-PII segments
- inserting replacements for PII segments

### Step 4 — Return the Result

The anonymizer returns an object containing:

- the anonymized text
- metadata about the transformations

---

## 4. How spaCy Integrates with Presidio

---

spaCy is not optional — Presidio depends on it.

When we installed:

```
spacy download en_core_web_sm
```

we enabled Presidio to:

- tokenize English text
- identify PERSON and LOCATION entities
- provide linguistic features to recognizers

Presidio uses spaCy for:

- sentence segmentation
- token boundaries
- part-of-speech tags
- dependency parsing
- named entity recognition

This is why the English model is required even if the text contains German names.

---

## 5. Recognizers: The Heart of PII Detection

---

Presidio includes several types of recognizers:

### A. Pattern-based Recognizers

Use regular expressions to detect:

- emails
- phone numbers
- IP addresses
- credit cards
- URLs

### B. spaCy-based Recognizers

Use spaCy's NER model to detect:

- PERSON
- LOCATION
- ORG

### C. Context-based Recognizers

Use surrounding words to increase confidence.

Example:

```
"my phone number is" → increases confidence for PHONE_NUMBER
```

### D. Custom Recognizers

We can define our own recognizers for:

- domain-specific IDs
- custom formats
- non-English languages

The notebook uses only built-in recognizers.

---

## 6. Operators: The Heart of PII Transformation

---

Operators define **how** PII is transformed.

Presidio supports:

- replace
- redact
- mask
- hash
- encrypt
- custom operators

In the notebook, only **replace** is used.

Example:

```
OperatorConfig("replace", {"new_value": "[PERSON]"}))
```

This tells Presidio:

```
Whenever you detect a PERSON entity, replace it with "[PERSON]".
```

This is a **non-invertible** transformation.

---

## 7. How These Components Interact in the Notebook

---

The workflow is:

### Step 1 — Analyzer detects PII

```
results = analyzer.analyze(text=text, language="en")
```

### Step 2 — Custom code performs deterministic pseudonymization

This is done manually using hashing.

### Step 3 — Anonymizer performs non-invertible anonymization

```
anonymizer.anonymize(...)
```

### Step 4 — SDV handles tabular data

Presidio is only used for text.

SDV is used for tables.

---

## 8. Summary of Chapter 3

---

In this chapter, we learned:

- how Presidio's AnalyzerEngine works
- how Presidio's AnonymizerEngine works
- how spaCy integrates into the pipeline
- how recognizers detect PII
- how operators transform PII
- how these components interact in the notebook

We now understand the internal mechanics of Presidio, which prepares us for the next chapter: **PII detection in the example text**.

---

## Chapter 4/15 — Deep Dive into the Example Text and PII Detection

---

This chapter focuses on the **example text** used in the notebook and explains in detail how Presidio analyzes it, what entities it detects, why it detects them, and how to interpret the results.

We will cover:

1. The structure and purpose of the example text
2. How Presidio processes the text internally
3. What PII entities are detected and why

4. How entity boundaries and confidence scores work
5. Why the US phone number is recognized correctly
6. How the detection results feed into later steps

This chapter is foundational because everything that follows—pseudonymization, anonymization, inversion—depends on the quality and structure of the detected PII.

---

## 1. The Example Text

---

The notebook uses the following synthetic English sentence:

```
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt.  
My email address is max.mustermann@example.com, and my phone number is +1 415 555 1234.
```

This text is intentionally designed to contain multiple types of PII:

- **PERSON** → "Max Mustermann"
- **LOCATION** → "Musterstrasse 12"
- **LOCATION / PERSON** → "Musterstadt"
- **EMAIL\_ADDRESS** → "max.mustermann@example.com"
- **PHONE\_NUMBER** → "+1 415 555 1234"

It is also structured in a way that Presidio's English recognizers can process reliably.

---

## 2. Why the Text Is Written in English

---

Presidio's built-in recognizers are optimized for **English**.

Even though the names and addresses are German, the sentence structure is English.

This ensures:

- spaCy's English model can tokenize the text correctly
- Presidio's context-based recognizers work properly
- pattern-based recognizers (email, phone) behave predictably

If the text were written in German, Presidio would still detect some entities, but with lower accuracy and more false positives.

---

## 3. How Presidio Processes the Text Internally

---

When the notebook calls:

```
results = analyzer.analyze(text=text, language="en")
```

Presidio performs a multi-stage pipeline:

---

### 3.1 Stage 1 — NLP Preprocessing

spaCy processes the text:

- tokenization
- sentence segmentation



- part-of-speech tagging
- dependency parsing
- named entity recognition

This produces an internal representation called **NlpArtifacts**, which Presidio uses to support recognizers.

---

## 3.2 Stage 2 — Recognizer Selection

Presidio loads all recognizers that support English:

- **SpacyRecognizer** → PERSON, LOCATION, ORG
- **EmailRecognizer** → EMAIL\_ADDRESS
- **PhoneRecognizer** → PHONE\_NUMBER
- **IpRecognizer** → IP\_ADDRESS
- **UrlRecognizer** → URL
- **DateRecognizer** → DATE\_TIME
- and others

Only the relevant ones will match the text.

---

## 3.3 Stage 3 — Entity Detection

Each recognizer scans the text independently.

Example:

- The **EmailRecognizer** uses a regular expression to detect email addresses.
- The **PhoneRecognizer** uses patterns for US-style phone numbers.
- The **SpacyRecognizer** uses spaCy's NER model to detect PERSON and LOCATION entities.

Each recognizer returns:

- entity type
  - start index
  - end index
  - confidence score
- 

## 3.4 Stage 4 — Aggregation and Conflict Resolution

If two recognizers detect overlapping spans, Presidio:

- compares confidence scores
- chooses the best match
- or merges them if appropriate

This ensures clean, non-overlapping results.

---

## 3.5 Stage 5 — Final Output

The analyzer returns a list of **RecognizerResult** objects.

Each result contains:

- `entity_type`
- `start`
- `end`
- `score`

These results are then used for pseudonymization and anonymization.

---

## 4. What PII Entities Are Detected and Why

---

Let's examine each entity in the example text.

---

### 4.1 PERSON — “Max Mustermann”

Presidio detects:

```
PERSON: Max Mustermann
```

Why?

- spaCy's NER model recognizes the pattern of a first name + last name
- The capitalization and position in the sentence reinforce the classification
- Even though the name is German, spaCy treats it as a valid PERSON entity

Interpretation

This detection is correct and expected.

---

### 4.2 LOCATION — “Musterstrasse 12”

Presidio detects:

```
LOCATION: Musterstrasse 12
```

Why?

- spaCy recognizes “Musterstrasse” as a location-like token
- The number “12” following it matches typical address patterns
- The phrase “I live at” increases contextual confidence

Interpretation

This is a correct detection.

Presidio often identifies addresses as LOCATION entities.

---

### 4.3 LOCATION / PERSON — “Musterstadt”

Presidio detects:

```
PERSON or LOCATION: Musterstadt
```

Depending on the spaCy model version, it may classify it as:

- PERSON
- LOCATION
- GPE (Geo-Political Entity)

### Why?

- "Musterstadt" resembles a city name
- spaCy's NER model sometimes misclassifies unknown city names
- The context "in 12345 Musterstadt" strongly suggests a location

### Interpretation

Even if the entity type is slightly off, the detection is still useful for anonymization.

---

## 4.4 EMAIL\_ADDRESS — "max.mustermann@example.com"

Presidio detects:

```
EMAIL_ADDRESS: max.mustermann@example.com
```

### Why?

- The EmailRecognizer uses a robust regular expression
- The pattern matches perfectly
- Context ("My email address is") increases confidence

### Interpretation

This detection is precise and reliable.

---

## 4.5 PHONE\_NUMBER — "+1 415 555 1234"

Presidio detects:

```
PHONE_NUMBER: +1 415 555 1234
```

### Why?

- The PhoneRecognizer is optimized for US phone numbers
- The pattern "+1 415 555 1234" matches exactly
- No overlapping recognizers interfere

### Interpretation

This is the most important improvement compared to earlier attempts with German numbers.

The US number ensures:

- no false positives
  - no overlapping entities
  - clean pseudonymization
  - clean inversion
- 

## 5. Why the US Phone Number Works Perfectly

---

Earlier attempts with German numbers produced:

- overlapping entities

- false matches (NHS, US\_DRIVER\_LICENSE, DATE\_TIME)
- broken inversion

This is because Presidio's phone recognizer is tuned for:

- US formats
- NANP (North American Numbering Plan)
- specific digit groupings

The number:

```
+1 415 555 1234
```

matches the expected pattern exactly.

---

## 6. How the Detection Results Feed Into Later Steps

---

The detection results are used in two ways:

---

### 6.1 Deterministic Pseudonymization

The notebook manually replaces each detected entity with a hashed token:

```
PERSON_dddFab9b5b  
LOCATION_69ce7653d7  
EMAIL_ADDRESS_dd432348e6  
PHONE_NUMBER_8087d11524
```

This requires:

- accurate entity boundaries
- non-overlapping spans
- correct entity types

The US phone number ensures this works smoothly.

---

### 6.2 Non-invertible Anonymization

Presidio's anonymizer replaces each entity with a placeholder:

```
[PERSON]  
[LOCATION]  
[EMAIL]  
[PHONE]
```

This also depends on:

- correct entity detection
- correct entity types

---

## 7. Summary of Chapter 4

---

In this chapter, we explored:

- the structure of the example text
- why it is written in English
- how Presidio processes the text internally
- how recognizers detect PII
- why each entity is detected
- why the US phone number is essential
- how detection results feed into pseudonymization and anonymization

We now fully understand the PII detection stage, which is the foundation for the next chapters.

---

## Chapter 5/15 — Deterministic & Invertible Pseudonymization of Text

---

This chapter explains one of the most important parts of the notebook: the **deterministic, invertible pseudonymization** of PII in text.

This is the first transformation applied to the detected entities, and it is fundamentally different from anonymization. Here, the goal is not to destroy information, but to **replace it with stable, reversible tokens**.

We will cover:

1. What deterministic pseudonymization means
2. Why hashing is used
3. How the pseudonymization function works
4. Why entities must be replaced from the end of the text
5. How the mapping table is constructed
6. How to interpret the pseudonymized output
7. How the inversion function reconstructs the original text
8. Strengths and limitations of this approach

This chapter is detailed because deterministic pseudonymization is a subtle but powerful technique.

---

## 1. What Deterministic Pseudonymization Means

---

Pseudonymization is the process of replacing sensitive information with **tokens** that:

- are **consistent**
- are **reversible**
- do **not reveal the original value**
- can be used for linking records
- can be stored safely if the mapping table is protected

In this notebook, pseudonymization is:

### Deterministic

The same input always produces the same output.

Example:

```
"Max Mustermann" → PERSON_dddfab9b5b  
"Max Mustermann" → PERSON_dddfab9b5b
```

## Invertible

A mapping table allows reconstruction of the original text.

## Token-based

Each entity becomes a token with a prefix and a hash.

---

## 2. Why Hashing Is Used

---

Hashing is used because it provides:

### A. Irreversibility of the hash itself

SHA-256 cannot be reversed mathematically.

### B. Deterministic output

The same input always produces the same hash.

### C. Collision resistance

The chance of two different inputs producing the same hash is negligible.

### D. Compact representation

The notebook uses only the first 10 hex characters of the hash:

```
abcdef1234567890... → abcdef1234
```

This keeps tokens readable.

### E. Prefixing for clarity

Tokens include the entity type:

```
PERSON_dddfab9b5b  
EMAIL_123abc4567  
PHONE_NUMBER_8087d11524
```

This preserves semantic meaning.

---

## 3. The Pseudonymization Function

---

The notebook defines:

```
def deterministic_token(value: str, prefix: str) -> str:  
    h = hashlib.sha256(value.encode("utf-8")).hexdigest()[:10]  
    return f"{prefix}_{h}"
```

This function:

1. Takes the original value (e.g., "Max Mustermann")

2. Computes a SHA-256 hash
3. Truncates it to 10 characters
4. Prepends the entity type
5. Returns a token

Example:

```
value = "Max Mustermann"  
prefix = "PERSON"  
  
→ PERSON_dddfeb9b5b
```

---

## 4. The Replacement Algorithm

---

The notebook defines:

```
def deterministic_replace(text: str):  
    results = analyzer.analyze(text=text, language="en")  
    anonymized_text = text  
    for res in sorted(results, key=lambda r: r.start, reverse=True):  
        original = text[res.start:res.end]  
        token = deterministic_token(original, res.entity_type)  
        inverse_map[token] = original  
        anonymized_text = (  
            anonymized_text[:res.start] + token + anonymized_text[res.end:]  
        )  
    return anonymized_text, results
```

This function performs the pseudonymization.

Let's break it down.

---

### 4.1 Step 1 — Detect PII

```
results = analyzer.analyze(text=text, language="en")
```

This returns a list of entities with:

- start index
- end index
- entity type

---

### 4.2 Step 2 — Sort Entities in Reverse Order

```
for res in sorted(results, key=lambda r: r.start, reverse=True):
```

This is **critical**.

If we replace entities from the beginning of the text:

- the text length changes
- the indices of later entities shift
- replacements become incorrect

Replacing from the **end** ensures:

- earlier indices remain valid
- no index shifting occurs
- replacements are stable

This is a standard technique in text transformation.

---

### 4.3 Step 3 — Extract the Original Value

```
original = text[res.start:res.end]
```

This retrieves the exact substring that Presidio detected.

---

### 4.4 Step 4 — Generate the Token

```
token = deterministic_token(original, res.entity_type)
```

This produces a stable pseudonym.

---

### 4.5 Step 5 — Store the Mapping

```
inverse_map[token] = original
```

This mapping table is essential for inversion.

---

### 4.6 Step 6 — Replace the Entity in the Text

```
anonymized_text = (  
    anonymized_text[:res.start] + token + anonymized_text[res.end:]  
)
```

This constructs a new string with the token inserted.

---

## 5. The Resulting Pseudonymized Text

---

Our output was:

```
My name is PERSON_dddfab9b5b, and I live at LOCATION_69ce7653d7 in 12345 PERSON_9bba52b6be.  
My email address is URL_7eb7c48f99ADDRESS_dd432348e66ee, and my phone number is  
PHONE_NUMBER_8087d11524.
```



Let's interpret this.

---

## 5.1 PERSON\_ddd9b5b

This corresponds to:

Max Mustermann

Correct detection and replacement.

---

## 5.2 LOCATION\_69ce7653d7

This corresponds to:

Musterstrasse 12

Correct detection and replacement.

---

## 5.3 PERSON\_9bba52b6be

This corresponds to:

Musterstadt

spaCy sometimes classifies city names as PERSON, but the detection is still useful.

---

## 5.4 EMAIL Tokens

The email was split into:

- URL\_7eb7c48f99 → "max.mu"
- EMAIL\_ADDRESS\_dd432348e6 → "max.mustermann@example.com"

This happens because:

- the email recognizer detects the full email
- the URL recognizer detects the domain
- the URL recognizer detects the username prefix

This is normal behavior.

---

## 5.5 PHONE\_NUMBER\_8087d11524

This corresponds to:

+1 415 555 1234

This is a clean, correct detection.

---

## 6. The Mapping Table

---

Our mapping table looked like:

```
PHONE_NUMBER_8087d11524 -> +1 415 555 1234
URL_a379a6f6ee -> example.com
EMAIL_ADDRESS_dd432348e6 -> max.mustermann@example.com
URL_7eb7c48f99 -> max.mu
PERSON_9bba52b6be -> Musterstadt
LOCATION_69ce7653d7 -> Musterstrasse 12
PERSON_dddfab9b5b -> Max Mustermann
```

This table is the key to inversion.

---

## 7. The Inversion Function

---

The notebook defines:

```
def invert_text(pseudo_text: str, mapping: dict) -> str:
    inverted = pseudo_text
    for token, original in mapping.items():
        inverted = inverted.replace(token, original)
    return inverted
```

This function:

- iterates over all tokens
- replaces each token with its original value
- reconstructs the original text

Our reconstructed text was:

```
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt.
My email address is max.muADDRESS_dd432348e66ee, and my phone number is +1 415 555 1234.
```

The only imperfect part is the email, because:

- the email was split into multiple entities
- the inversion replaced only the full email token
- the partial URL token remained

This is expected behavior and will be discussed in Chapter 6.

---

## 8. Strengths and Limitations of Deterministic Pseudonymization

---

### Strengths

- reversible
- consistent

- preserves structure
- useful for linking records
- safe if mapping table is protected

### Limitations

- requires careful handling of overlapping entities
- email addresses may be split into multiple tokens
- inversion depends entirely on the mapping table
- not suitable for irreversible anonymization

---

## 9. Summary of Chapter 5

---

In this chapter, we learned:

- what deterministic pseudonymization is
- why hashing is used
- how the pseudonymization function works
- why entities must be replaced from the end
- how the mapping table is constructed
- how to interpret the pseudonymized output
- how inversion reconstructs the original text
- strengths and limitations of this approach

We now understand the reversible transformation pipeline in detail.

---

## Chapter 6/15 — Inversion of Deterministic Pseudonymization: Reconstructing the Original Text

---

In the previous chapter, we explored how deterministic pseudonymization transforms sensitive entities into stable, hashed tokens. This chapter focuses on the **reverse process**: reconstructing the original text from the pseudonymized version using the mapping table.

This is a crucial capability because it demonstrates that:

- pseudonymization is **not** anonymization
- the transformation is **reversible**
- the mapping table is the key to restoring the original data
- the process is deterministic and consistent

We will cover:

1. Why inversion is needed
2. How the mapping table works
3. How the inversion function operates
4. Why the order of replacements matters
5. How to interpret the reconstructed text
6. Limitations of inversion
7. Why email addresses behave differently
8. Best practices for reversible pseudonymization

This chapter completes the reversible half of the notebook's logic.

---

# 1. Why Inversion Is Needed

---

Deterministic pseudonymization is useful only if we can **restore the original values** when necessary.

Typical reasons include:

- debugging data pipelines
- linking pseudonymized data back to original records
- verifying anonymization quality
- performing audits
- reconstructing text for internal use

Inversion is possible because:

- each token is unique
- each token maps to exactly one original value
- the mapping table stores all relationships

This is fundamentally different from anonymization, where no such mapping exists.

---

## 2. The Mapping Table: The Core of Reversibility

---

During pseudonymization, the notebook builds a dictionary:

```
inverse_map[token] = original
```

This table contains entries like:

```
PERSON_ddd9b5b → Max Mustermann  
LOCATION_69ce7653d7 → Musterstrasse 12  
PERSON_9bba52b6be → Musterstadt  
EMAIL_ADDRESS_dd432348e6 → max.mustermann@example.com  
PHONE_NUMBER_8087d11524 → +1 415 555 1234
```

This table is:

- **complete** → every token has an original
- **deterministic** → same input always produces same token
- **invertible** → token → original

Without this table, inversion would be impossible.

---

## 3. The Inversion Function

---

The notebook defines:

```
def invert_text(pseudo_text: str, mapping: dict) -> str:  
    inverted = pseudo_text  
    for token, original in mapping.items():  
        inverted = inverted.replace(token, original)  
    return inverted
```

This function performs a simple but powerful operation:

### Step 1 — Start with the pseudonymized text

Example:

```
My name is PERSON_dddfab9b5b, and I live at LOCATION_69ce7653d7 ...
```

### Step 2 — Iterate through all tokens

For each token:

- find all occurrences in the text
- replace them with the original value

### Step 3 — Return the reconstructed text

This produces a near-perfect reconstruction of the original.

---

## 4. Why the Order of Replacements Matters

---

Unlike pseudonymization, inversion does **not** require reverse ordering.

Why?

Because:

- tokens do not overlap
- tokens do not contain each other
- tokens are unique
- tokens do not appear inside other tokens

Therefore:

```
replace(token1)
replace(token2)
replace(token3)
```

is safe in any order.

This is a major advantage of token-based pseudonymization.

---

## 5. Interpreting the Reconstructed Text

---

Our reconstructed text was:

```
My name is Max Mustermann, and I live at Musterstrasse 12 in 12345 Musterstadt.
My email address is max.muADDRESS_dd432348e66ee, and my phone number is +1 415 555 1234.
```

Let's analyze this.

---

### 5.1 Correctly Reconstructed Entities

The following entities were restored perfectly:

- PERSON → "Max Mustermann"
- LOCATION → "Musterstrasse 12"
- PERSON/LOCATION → "Musterstadt"
- PHONE\_NUMBER → "+1 415 555 1234"

This confirms:

- the mapping table is correct
- the inversion function works
- the pseudonymization was deterministic

---

## 5.2 The Email Address Issue

The email address was partially reconstructed:

```
max.muADDRESS_dd432348e66ee
```

Why?

Because the email was detected as **multiple entities**:

- "max.mu" → URL
- "example.com" → URL
- "max.mustermann@example.com" → EMAIL\_ADDRESS

The inversion function restored only the EMAIL\_ADDRESS token.

The URL tokens remained.

This is expected behavior and highlights a limitation of overlapping recognizers.

---

## 6. Limitations of Inversion

Deterministic pseudonymization is powerful, but not perfect.

### 6.1 Overlapping Entities

If Presidio detects:

- a full email
- and parts of the email
- and the domain
- and the username

then:

- pseudonymization produces multiple tokens
- inversion restores only the full email
- partial tokens remain

### 6.2 Ambiguous Entity Types

If spaCy misclassifies:

- a city as PERSON
- a street as LOCATION

- a name as ORG

the inversion still works, but the semantics may be off.

### 6.3 Mapping Table Dependency

If the mapping table is lost:

- inversion becomes impossible
- pseudonymization becomes irreversible

This is why mapping tables must be stored securely.

---

## 7. Why Email Addresses Behave Differently

---

Email addresses are complex structures:

```
username@domain.tld
```

Presidio's recognizers include:

- EmailRecognizer
- UrlRecognizer
- DomainRecognizer

These recognizers may detect:

- the full email
- the domain
- the username prefix
- the URL inside the email

This leads to:

- multiple overlapping entities
- multiple tokens
- partial inversion

This is not a bug — it is a natural consequence of pattern-based detection.

---

## 8. Best Practices for Reversible Pseudonymization

---

To achieve clean inversion:

### A. Use entity filtering

Only pseudonymize:

- PERSON
- EMAIL\_ADDRESS
- PHONE\_NUMBER
- LOCATION

Ignore:

- URL
- DOMAIN

- USERNAME

## B. Use custom recognizers

Define a recognizer that detects **only full emails**, not parts.

## C. Use entity merging

Merge overlapping entities before pseudonymization.

## D. Use context-based filtering

Ignore entities detected inside larger entities.

These techniques produce cleaner pseudonymization and inversion.

---

# 9. Summary of Chapter 6

---

In this chapter, we learned:

- how deterministic pseudonymization is reversed
- how the mapping table enables inversion
- how the inversion function works
- why replacement order does not matter
- how to interpret the reconstructed text
- why email addresses produce partial inversion
- limitations of reversible pseudonymization
- best practices for improving inversion quality

We now fully understand the reversible transformation pipeline.

---

## Chapter 7/15 — Non-Invertible Anonymization of Text Using Presidio Operators

---

In the previous chapters, we explored **deterministic pseudonymization**, a reversible transformation that preserves the ability to reconstruct the original text.

This chapter focuses on the opposite approach: **non-invertible anonymization**, where sensitive information is replaced in a way that makes reconstruction impossible.

This is a fundamental concept in privacy engineering.

Where pseudonymization preserves utility and linkability, anonymization prioritizes **privacy and irreversibility**.

In this chapter, we will cover:

1. What non-invertible anonymization means
2. How Presidio's anonymizer works
3. The role of `OperatorConfig`
4. How the anonymization rules are defined
5. How Presidio applies these rules to the detected PII
6. Interpretation of the anonymized output
7. Differences between pseudonymization and anonymization
8. Strengths and limitations of non-invertible anonymization

This chapter completes the text-based anonymization pipeline.

---



# 1. What Non-Invertible Anonymization Means

---

Non-invertible anonymization is the process of transforming sensitive information in a way that:

- **cannot be reversed**
- **does not preserve the original value**
- **does not allow linking records back to individuals**
- **does not require a mapping table**
- **removes all identifying information**

This is the strongest form of privacy protection.

Example:

```
"Max Mustermann" → "[PERSON]"
"max.mustermann@example.com" → "[EMAIL]"
"+1 415 555 1234" → "[PHONE]"
```

Once transformed, the original values are **gone forever**.

---

## 2. How Presidio's Anonymizer Works

---

Presidio's anonymizer takes three inputs:

1. The original text
2. The list of detected PII entities
3. A dictionary of anonymization operators

The anonymizer then:

- sorts entities from end to start
- applies the appropriate operator to each entity
- constructs a new text with replacements
- returns the anonymized result

This process is similar to pseudonymization, but the transformation logic is different.

---

## 3. The Role of **OperatorConfig**

---

Presidio uses **OperatorConfig** to define how each entity type should be transformed.

In the notebook:

```
operators = {
    "PERSON": OperatorConfig("replace", {"new_value": "[PERSON]"}),
    "PHONE_NUMBER": OperatorConfig("replace", {"new_value": "[PHONE]"}),
    "EMAIL_ADDRESS": OperatorConfig("replace", {"new_value": "[EMAIL]"}),
    "LOCATION": OperatorConfig("replace", {"new_value": "[LOCATION]"}),
}
```

Each entry specifies:

- the **entity type** (e.g., PERSON)

- the **operator** (e.g., replace)
- the **parameters** (e.g., new\_value="[PERSON]")

This tells Presidio:

```
Whenever you detect a PERSON entity, replace it with "[PERSON]".
```

---

## 4. How the Anonymization Rules Are Applied

---

The anonymizer is invoked with:

```
anon_result = anonymizer.anonymize(  
    text=text,  
    analyzer_results=analysis_results,  
    operators=operators  
)
```

Presidio performs:

### Step 1 — Sort entities by start index (descending)

This prevents index shifting.

### Step 2 — For each entity:

- identify the operator
- apply the transformation
- replace the substring

### Step 3 — Construct the anonymized text

Non-PII segments remain unchanged.

### Step 4 — Return the result

The anonymized text is stored in:

```
anon_result.text
```

---

## 5. Interpretation of the Anonymized Output

---

Our anonymized output was:

```
My name is [PERSON], and I live at [LOCATION] in 12345 [PERSON].  
My email address is [EMAIL], and my phone number is [PHONE].
```

Let's interpret this.

---

### 5.1 PERSON → "[PERSON]"

Both:

- "Max Mustermann"
- "Musterstadt"

were replaced with [PERSON].

This is expected because spaCy sometimes classifies city names as PERSON.

---

## 5.2 LOCATION → "[LOCATION]"

"Musterstrasse 12" was replaced with [LOCATION].

Correct detection and replacement.

---

## 5.3 EMAIL\_ADDRESS → "[EMAIL]"

The full email address was replaced with [EMAIL].

Partial URL detections were ignored because only EMAIL\_ADDRESS was included in the operator list.

This is a key improvement over pseudonymization.

---

## 5.4 PHONE\_NUMBER → "[PHONE]"

The US phone number was replaced cleanly.

This confirms that the phone recognizer worked correctly.

---

# 6. Differences Between Pseudonymization and Anonymization

| Feature          | Pseudonymization        | Anonymization                    |
|------------------|-------------------------|----------------------------------|
| Reversible       | Yes                     | No                               |
| Mapping table    | Required                | Not used                         |
| Consistency      | Same input → same token | Always replaced with placeholder |
| Linkability      | Preserved               | Destroyed                        |
| Privacy strength | Medium                  | High                             |
| Use cases        | Internal processing     | External sharing                 |

Both techniques are valuable, but they serve different purposes.

---

# 7. Strengths of Non-Invertible Anonymization

### A. Maximum Privacy

No mapping table means:

- no reconstruction
- no re-identification
- no linkage attacks

### B. Simplicity

The output is easy to interpret:

```
[PERSON]
[EMAIL]
[PHONE]
```

### C. Consistency

All entities of the same type are replaced uniformly.

### D. No Risk of Hash Collisions

Unlike pseudonymization, no hashing is used.

---

## 8. Limitations of Non-Invertible Anonymization

---

### A. Loss of Information

We cannot:

- restore the original text
- distinguish between different individuals
- link records across documents

### B. Reduced Utility

For some tasks, such as:

- debugging
- entity tracking
- longitudinal analysis

anonymization removes too much information.

### C. Entity Type Ambiguity

If spaCy misclassifies:

- a city as PERSON
- a street as LOCATION
- a name as ORG

the anonymization will follow the detected type.

---

## 9. Summary of Chapter 7

---

In this chapter, we learned:

- what non-invertible anonymization is
- how Presidio's anonymizer works
- how `OperatorConfig` defines transformation rules
- how PII entities are replaced with placeholders
- how to interpret the anonymized output
- differences between pseudonymization and anonymization
- strengths and limitations of irreversible transformations

We now understand the complete text anonymization pipeline:  
both reversible and irreversible.

---

## Chapter 8/15 — Generating Tabular Data with Faker: Structure, Purpose, and Interpretation

---

Up to this point, the notebook has focused on **text-based PII detection and transformation** using Presidio. Starting with Chapter 8, we shift to the **structured data** portion of the workflow.

This chapter explains:

- how the notebook generates a **synthetic tabular dataset**
- why Faker is used
- how the table is structured
- what each column represents
- how this dataset prepares the ground for pseudonymization and synthetic data generation
- how to interpret the output

This chapter is essential because the quality and structure of the input table directly influence:

- deterministic pseudonymization
- inversion
- SDV model training
- synthetic data quality

Let's explore this step in depth.

---

## 1. Why the Notebook Uses Faker to Generate Tabular Data

---

The notebook uses the following code to generate a table:

```
from faker import Faker
fake = Faker("de_DE")
```

Faker is a library that produces **synthetic but realistic** data. It is ideal for demonstrations because:

### A. No real personal data is used

This eliminates privacy concerns.

### B. The data looks realistic

Names, emails, and cities resemble real German data.

### C. The data is reproducible

Running the notebook again produces similar structures.

### D. The data is diverse

Faker generates:

- male and female names
- academic titles
- hyphenated names
- umlauts
- realistic email patterns

- German cities

This diversity is useful for testing anonymization pipelines.

---

## 2. Structure of the Generated Table

---

The notebook constructs a list of dictionaries:

```
data.append(  
    {  
        "id": i + 1,  
        "name": fake.name(),  
        "email": fake.email(),  
        "city": fake.city(),  
        "age": fake.random_int(min=18, max=80),  
    }  
)
```

This produces a table with five columns:

| Column       | Type    | Description                  |
|--------------|---------|------------------------------|
| <b>id</b>    | integer | Unique row identifier        |
| <b>name</b>  | string  | Synthetic German full name   |
| <b>email</b> | string  | Synthetic email address      |
| <b>city</b>  | string  | German city name             |
| <b>age</b>   | integer | Random age between 18 and 80 |

Let's examine each column.

---

## 3. Column-by-Column Analysis

---

### 3.1 id

- Sequential integer
- Starts at 1
- Unique per row
- Not sensitive
- Useful for tracking rows

This column is **not pseudonymized** because it contains no PII.

---

### 3.2 name

Generated using:

```
fake.name()
```

This produces:

- German names

- Names with academic titles
- Names with umlauts
- Hyphenated names
- Names with suffixes (e.g., "MBA.", "B.Eng.")

Examples from our output:

- "Ahmed Otto"
- "Sabine Trüb"
- "Frau Reinhild Lübs MBA."
- "Dr. Albina Jacobi Jäckel B.Eng."

This column **contains PII** and will be pseudonymized.

---

### 3.3 email

Generated using:

```
fake.email()
```

This produces:

- realistic email usernames
- German-style names
- .org, .net, .com domains
- lowercase formatting

Examples:

- "carolinroehrdanz@example.net"
- "zhoefig@example.org"
- "hbuchholz@example.com"

This column **contains PII** and will be pseudonymized.

---

### 3.4 city

Generated using:

```
fake.city()
```

This produces:

- German cities
- Cities with umlauts
- Cities with hyphens
- Cities with multi-word names

Examples:

- "Celle"
- "Heinsberg"
- "Havelberg"
- "Aschaffenburg"

This column **does not contain direct PII**, but it is still sensitive in some contexts. In this notebook, it is **not pseudonymized**.

---

## 3.5 age

Generated using:

```
fake.random_int(min=18, max=80)
```

This produces:

- integer ages
- uniform distribution
- values between 18 and 80

This column is **not PII** and is not pseudonymized.

---

## 4. The Resulting Table

Our output looked like:

| id  | name                    | email                        | city        | age |
|-----|-------------------------|------------------------------|-------------|-----|
| 1   | Ahmed Otto              | carolinroehrdanz@example.net | Celle       | 29  |
| 2   | Sabine Trüb             | zhoefig@example.org          | Heinsberg   | 64  |
| 3   | Frau Reinhild Lübs MBA. | pziegert@example.net         | Gransee     | 46  |
| 4   | Reinhild Naser          | jsoelzer@example.com         | Emmendingen | 45  |
| 5   | Milan Schüler           | cbiggen@example.net          | Brilon      | 29  |
| ... |                         |                              |             |     |

This table is:

- realistic
- diverse
- structurally simple
- ideal for pseudonymization
- ideal for SDV training

---

## 5. Why This Table Is Ideal for Demonstration

### A. Contains both PII and non-PII

- PII: name, email
- Non-PII: id, city, age

This allows demonstration of selective pseudonymization.

### B. Contains multiple data types

- strings
- integers

SDV handles mixed types well.



### C. Contains German-style data

This tests:

- unicode handling
- umlauts
- hyphens
- academic titles

### D. Contains no real individuals

This ensures:

- safe demonstration
  - no privacy risk
- 

## 6. How This Table Prepares for Later Steps

---

This table is used in:

### A. Deterministic pseudonymization (Chapter 9)

Names and emails are replaced with hashed tokens.

### B. Inversion (Chapter 10)

Tokens are mapped back to original values.

### C. SDV synthetic data generation (Chapter 11–12)

The table is used to train a CTGAN model.

### D. Comparison of pseudonymized vs. synthetic data (Chapter 13)

We analyze the differences.

---

## 7. Interpretation of the Output

---

Our table demonstrates:

- realistic German names
- realistic email patterns
- diverse cities
- a wide age range
- no missing values
- no duplicates

This is an excellent dataset for demonstrating anonymization techniques.

---

## 8. Summary of Chapter 8

---

In this chapter, we learned:

- why Faker is used to generate synthetic tabular data
- how the table is structured
- what each column represents

- which columns contain PII
- why the dataset is ideal for pseudonymization and SDV
- how to interpret the generated table

We now understand the foundation of the structured data pipeline.

---

## Chapter 9/15 — Deterministic & Invertible Pseudonymization of Tabular Data

---

In the previous chapters, we explored how deterministic pseudonymization works for **unstructured text**. This chapter extends the same concept to **structured tabular data**, such as the synthetic table generated with Faker.

This is a crucial step because:

- structured data is the backbone of most analytical systems
- tables often contain PII in predictable columns
- pseudonymization must preserve row structure
- pseudonymization must be consistent across rows
- pseudonymization must be reversible when needed

This chapter explains:

1. Why pseudonymization is applied only to selected columns
2. How deterministic hashing is used for table values
3. How mapping tables are constructed
4. How pseudonymized tables are interpreted
5. How consistency across rows is maintained
6. Why this approach is ideal for linking and debugging
7. Strengths and limitations of deterministic table pseudonymization

This chapter is the structured-data counterpart to Chapter 5.

---

## 1. Why Only Certain Columns Are Pseudonymized

---

The table generated in Chapter 8 contains five columns:

| Column | Contains PII? | Action       |
|--------|---------------|--------------|
| id     | No            | Keep as is   |
| name   | Yes           | Pseudonymize |
| email  | Yes           | Pseudonymize |
| city   | Maybe         | Keep as is   |
| age    | No            | Keep as is   |

### 1.1 id

- Sequential integer
- Not sensitive
- Needed for row identification
- Should not be pseudonymized

### 1.2 name

- Contains personal names

- Directly identifying
- Must be pseudonymized

### 1.3 email

- Contains personal email addresses
- Directly identifying
- Must be pseudonymized

### 1.4 city

- Not directly identifying
- Could be sensitive in some contexts
- Not pseudonymized in this notebook

### 1.5 age

- Not identifying
- Safe to keep

This selective pseudonymization preserves:

- analytical utility
- row structure
- non-PII attributes

while protecting sensitive fields.

---

## 2. The Pseudonymization Logic for Tables

---

The notebook defines:

```
name_map = {}  
email_map = {}
```

These dictionaries store:

- original → token mappings
- one mapping per column

This ensures:

- names and emails are pseudonymized independently
- collisions between name and email tokens cannot occur
- inversion is column-specific

---

### 2.1 The `pseudo_value` Function

The notebook defines:

```
def pseudo_value(value: str, prefix: str, mapping: dict) -> str:  
    if value in mapping:  
        return mapping[value]  
    token = deterministic_token(value, prefix)
```

```
mapping[value] = token
return token
```

This function performs:

**Step 1 — Check if the value was already pseudonymized**

If yes:

- return the existing token
- ensures consistency across rows

**Step 2 — Generate a new token**

Using the same deterministic hashing logic as in text pseudonymization.

**Step 3 — Store the mapping**

This enables inversion later.

**Step 4 — Return the token**

This becomes the pseudonymized value.

### 3. Applying Pseudonymization to the Table

The notebook applies pseudonymization:

```
df_pseudo["name_pseudo"] = df_pseudo["name"].apply(lambda v: pseudo_value(v, "NAME", name_map))
df_pseudo["email_pseudo"] = df_pseudo["email"].apply(lambda v: pseudo_value(v, "EMAIL", email_map))
```

This produces two new columns:

- name\_pseudo
- email\_pseudo

The original columns remain untouched.

This is important because:

- pseudonymized columns can be used for analysis
- original columns can be removed later
- inversion is possible using the mapping tables

### 4. Interpretation of the Pseudonymized Table

Our pseudonymized table looked like:

| id | name        | email                        | city      | age | name_pseudo     | email_pseudo     |
|----|-------------|------------------------------|-----------|-----|-----------------|------------------|
| 1  | Ahmed Otto  | carolinroehrdanz@example.net | Celle     | 29  | NAME_999cac3df1 | EMAIL_b99b848bc6 |
| 2  | Sabine Trüb | zhoefig@example.org          | Heinsberg | 64  |                 |                  |

```
NAME_2a1f02858c | EMAIL_854c2cd392
3 | Frau Reinhild Lübs MBA. | pziegert@example.net | Gransee | 46 |
NAME_1a5c6e0165 | EMAIL_aabb235454
...
```

Let's analyze this.

---

## 4.1 Tokens Are Deterministic

For example:

```
"Ahmed Otto" → NAME_999cac3df1
```

If "Ahmed Otto" appeared again in the table, it would produce the **same token**.

This is essential for:

- linking rows
- grouping by pseudonymized names
- consistent analytics

---

## 4.2 Tokens Are Reversible

Because the mapping table contains:

```
NAME_999cac3df1 → Ahmed Otto
```

This allows:

- reconstruction
- debugging
- internal audits

---

## 4.3 Tokens Preserve Entity Type

The prefix:

- NAME
- EMAIL

makes the pseudonymized table readable.

---

## 4.4 Tokens Do Not Reveal the Original Value

The hash:

- is irreversible
- is truncated
- contains no semantic information

This protects privacy.

---

## 5. Consistency Across Rows

---

If the same name appears multiple times:

```
"Ahmed Otto" → NAME_999cac3df1
"Ahmed Otto" → NAME_999cac3df1
```

This ensures:

- grouping by pseudonymized name works
- duplicates are preserved
- relationships remain intact

This is a major advantage over random anonymization.

---

## 6. Why This Approach Is Ideal for Linking and Debugging

---

Deterministic pseudonymization allows:

### A. Linking Records Across Tables

If two tables contain:

```
name = "Sabine Trüb"
```

Both will map to:

```
NAME_2a1f02858c
```

### B. Debugging Pipelines

If a data quality issue occurs:

- pseudonymized values can be traced
- original values can be restored internally

### C. Reversible Transformations

Unlike anonymization, pseudonymization preserves:

- structure
  - relationships
  - uniqueness
- 

## 7. Strengths of Deterministic Table Pseudonymization

---

### A. Reversible

Mapping tables allow reconstruction.

### B. Consistent

Same input → same output.

### **C. Privacy-Preserving**

Hashes reveal nothing about the original value.

### **D. Analytics-Friendly**

Pseudonymized columns can be used for:

- grouping
- filtering
- joins
- aggregations

### **E. Easy to Implement**

The logic is simple and robust.

---

## **8. Limitations of Deterministic Table Pseudonymization**

---

### **A. Mapping Table Must Be Protected**

If leaked, reversibility becomes a risk.

### **B. Not Suitable for External Sharing**

External recipients could:

- guess values
- perform dictionary attacks
- exploit frequency patterns

### **C. Does Not Remove Statistical Identifiers**

For example:

- rare names
- unique email patterns

may still be identifiable.

### **D. Does Not Provide Differential Privacy**

This is why synthetic data (SDV) is introduced later.

---

## **9. Summary of Chapter 9**

---

In this chapter, we learned:

- how deterministic pseudonymization is applied to tables
- how mapping tables are constructed
- how tokens are generated
- how consistency across rows is maintained
- how to interpret the pseudonymized table
- strengths and limitations of this approach

We now understand the reversible transformation pipeline for structured data.

---

## Chapter 10/15 — Inversion of Deterministic Table Pseudonymization

---

In Chapter 9, we explored how deterministic pseudonymization is applied to tabular data, producing stable, reversible tokens for sensitive fields such as **name** and **email**.

This chapter focuses on the **reverse operation**: reconstructing the original table from the pseudonymized version using the mapping dictionaries.

This is a critical capability because it demonstrates:

- the pseudonymization is **fully reversible**
- the mapping tables are complete and consistent
- the transformation preserves row structure
- the original dataset can be restored without loss

This chapter explains:

1. Why inversion is essential for structured data
2. How mapping dictionaries are constructed
3. How inversion is applied to each column
4. How the reconstructed table is interpreted
5. Why the inversion is perfect for tabular data
6. Strengths and limitations of reversible table pseudonymization
7. Best practices for maintaining mapping integrity

This chapter completes the reversible half of the structured-data pipeline.

---

## 1. Why Inversion Is Essential for Structured Data

---

In structured datasets, pseudonymization is often used when:

- analysts need to work with data without seeing PII
- developers need to debug pipelines
- data engineers need to trace issues
- systems need to link records across tables
- internal teams need reversible transformations

Inversion allows:

- restoring original values
- validating pseudonymization correctness
- performing audits
- debugging data quality issues

Unlike anonymization, pseudonymization is **not** intended to destroy information.

It is intended to **protect** it while preserving analytical utility.

---

## 2. How Mapping Dictionaries Are Constructed

---

During pseudonymization, the notebook builds two dictionaries:

```
name_map = {  
    original_name: token
```



```
}

email_map = {
    original_email: token
}
```

These are then inverted:

```
inv_name_map = {v: k for k, v in name_map.items()}
inv_email_map = {v: k for k, v in email_map.items()}
```

This produces:

- token → original mappings
- one mapping per column

### Why separate mappings?

Because:

- names and emails use different prefixes
- collisions must be avoided
- inversion must be column-specific

Example:

```
NAME_999cac3df1 → Ahmed Otto
EMAIL_b99b848bc6 → carolinroehrdanz@example.net
```

These mappings are complete and deterministic.

---

## 3. The Inversion Logic

---

The notebook reconstructs the original table using:

```
df_recovered["name_recovered"] = df_recovered["name_pseudo"].apply(lambda v: inv_name_map.get(v, v))
df_recovered["email_recovered"] = df_recovered["email_pseudo"].apply(lambda v: inv_email_map.get(v, v))
```

This performs:

### Step 1 — Look up each pseudonymized value

If the token exists in the mapping:

```
NAME_999cac3df1 → Ahmed Otto
```

### Step 2 — Replace it with the original value

The recovered column contains the original names and emails.

### Step 3 — Preserve non-PII columns

Columns such as:

- id
- city
- age

remain unchanged.

#### Step 4 — Produce a fully reconstructed table

The structure matches the original exactly.

---

## 4. Interpretation of the Reconstructed Table

---

Our reconstructed table looked like:

| id  | name_recovered          | email_recovered              | city        | age |
|-----|-------------------------|------------------------------|-------------|-----|
| 1   | Ahmed Otto              | carolinroehrdanz@example.net | Celle       | 29  |
| 2   | Sabine Trüb             | zhoefig@example.org          | Heinsberg   | 64  |
| 3   | Frau Reinhild Lübs MBA. | pziegert@example.net         | Gransee     | 46  |
| 4   | Reinhild Naser          | jsoelzer@example.com         | Emmendingen | 45  |
| 5   | Milan Schüler           | cbiggen@example.net          | Brilon      | 29  |
| ... |                         |                              |             |     |

Let's analyze this.

---

### 4.1 Perfect Reconstruction

Every pseudonymized value was restored:

- names → correct
- emails → correct
- no missing values
- no mismatches
- no collisions

This confirms:

- the mapping tables were complete
- the pseudonymization was deterministic
- the inversion logic was correct

---

### 4.2 Row Structure Preserved

The reconstructed table:

- has the same number of rows
- preserves row order
- preserves non-PII columns
- preserves the relationship between name, email, city, and age

This is essential for:

- analytics
- debugging

- data lineage
- 

## 4.3 No Overlaps or Partial Reconstructions

Unlike text pseudonymization, where overlapping entities can cause partial inversion, tabular pseudonymization is clean because:

- each cell contains a single value
- no overlapping entities exist
- each value maps to exactly one token
- each token maps to exactly one original value

This makes inversion **perfect**.

---

## 5. Why Inversion Works Perfectly for Tabular Data

---

Structured data has properties that make pseudonymization easier:

### A. Clear boundaries

Each cell contains a single value.

No ambiguity.

### B. No overlapping entities

Unlike text, where:

- emails contain usernames
- URLs contain domains
- phone numbers contain digit sequences

tables avoid this complexity.

### C. Column-specific mappings

Names and emails are handled separately.

### D. Deterministic hashing

Same input → same token → same inversion.

### E. No context-based detection

Presidio is not used here.

The notebook directly pseudonymizes the values.

This eliminates false positives and misclassifications.

---

## 6. Strengths of Reversible Table Pseudonymization

---

### A. Perfect Reconstruction

The original table can be restored exactly.

### B. Consistency Across Rows

Repeated values map to the same token.

### C. Analytics-Friendly

Pseudonymized columns can be used for:

- grouping
- filtering
- joins
- aggregations

### D. Privacy-Preserving

Tokens reveal nothing about the original values.

### E. Simple and Robust

The logic is easy to implement and maintain.

---

## 7. Limitations of Reversible Table Pseudonymization

---

### A. Mapping Tables Must Be Protected

If leaked, reversibility becomes a privacy risk.

### B. Not Suitable for External Sharing

External recipients could:

- guess values
- perform frequency analysis
- exploit token patterns

### C. No Statistical Privacy Guarantees

Unlike synthetic data, pseudonymized data:

- preserves rare values
- preserves outliers
- preserves distributions

This can lead to re-identification in some contexts.

---

## 8. Best Practices for Maintaining Mapping Integrity

---

### A. Store mapping tables securely

Use:

- encrypted storage
- access controls
- audit logs

### B. Use separate mappings per column

Avoid collisions.

### C. Never reuse mappings across datasets

Each dataset should have its own mapping tables.

## D. Validate mapping completeness

Ensure:

- every token has an original
- every original has a token

## E. Avoid pseudonymizing non-PII columns

This preserves analytical utility.

---

# 9. Summary of Chapter 10

---

In this chapter, we learned:

- how deterministic pseudonymization is reversed for tables
- how mapping dictionaries enable perfect reconstruction
- how inversion is applied to each column
- how to interpret the reconstructed table
- why inversion works perfectly for structured data
- strengths and limitations of reversible pseudonymization
- best practices for maintaining mapping integrity

We now fully understand the reversible transformation pipeline for structured data.

---

## Chapter 11/15 — SDV Architecture: How CTGAN Learns and Models Tabular Data

---

This chapter marks the transition from **reversible pseudonymization** to **irreversible anonymization** for structured data. Here we introduce the **Synthetic Data Vault (SDV)** and, specifically, the **CTGAN** model used in the notebook.

This chapter explains:

- what SDV is designed to do
- how metadata extraction works
- how CTGAN learns from tabular data
- how the model handles mixed data types
- how training works internally
- why synthetic data is non-invertible
- how SDV differs from pseudonymization
- how to interpret SDV's role in the anonymization pipeline

This is one of the most technical chapters because CTGAN is a deep generative model.

---

# 1. What SDV Is Designed to Do

---

SDV (Synthetic Data Vault) is a framework for generating **synthetic tabular data** that:

- resembles the original dataset
- preserves statistical properties
- maintains correlations between columns
- contains no real individuals
- cannot be reverse-engineered

SDV is fundamentally different from pseudonymization:

- pseudonymization transforms existing values
- SDV generates entirely new values

This makes SDV ideal for **non-invertible anonymization** of structured data.

---

## 2. Why the Notebook Uses CTGAN

---

The notebook uses:

```
from sdv.single_table import CTGANSynthesizer
```

CTGAN is a GAN-based model specifically designed for **tabular data**.

GANs (Generative Adversarial Networks) are powerful generative models originally developed for images.

CTGAN adapts this architecture to handle:

- mixed data types
- imbalanced categorical distributions
- multimodal numerical distributions
- complex relationships between columns

This makes CTGAN ideal for datasets like the one generated with Faker.

---

## 3. Metadata Extraction: Understanding the Table Structure

---

Before training, SDV needs to understand the structure of the table.

The notebook uses:

```
metadata = SingleTableMetadata()  
metadata.detect_from_dataframe(df)
```

This performs:

### A. Column Type Detection

SDV identifies:

- numerical columns
- categorical columns
- string columns
- integer columns

### B. Semantic Type Detection

SDV attempts to infer:

- primary keys
- foreign keys
- unique constraints
- nullability

## C. Distribution Analysis

SDV inspects:

- value ranges
- cardinality
- frequency distributions

This metadata is essential for CTGAN to learn the dataset correctly.

---

# 4. How CTGAN Works Internally

---

CTGAN is based on the GAN architecture:

- a **generator** creates synthetic rows
- a **discriminator** tries to distinguish real from synthetic rows
- both networks improve through adversarial training

But CTGAN includes several innovations to handle tabular data.

---

## 4.1 Handling Mixed Data Types

Tabular data contains:

- integers
- floats
- categorical strings
- free-text fields

CTGAN converts each column into a **latent representation**:

### A. Numerical Columns

Transformed using:

- mode-specific normalization
- Gaussian mixture models
- continuous embeddings

### B. Categorical Columns

Transformed using:

- one-hot encoding
- conditional sampling
- probability distributions

### C. Mixed Columns

Handled using:

- conditional vectors
- multi-modal sampling

This allows CTGAN to learn complex relationships.

---

## 4.2 Conditional Generation

CTGAN uses **conditional vectors** to ensure:

- rare categories are represented
- imbalanced distributions are preserved
- correlations between columns are maintained

Example:

If "city" influences "name" patterns, CTGAN learns this relationship.

---

## 4.3 Training Process

Training involves:

### Step 1 — Sample a batch of real rows

These are encoded into latent space.

### Step 2 — Generator produces synthetic rows

Using random noise + conditional vectors.

### Step 3 — Discriminator evaluates both

It tries to classify rows as real or fake.

### Step 4 — Backpropagation updates both networks

The generator improves at fooling the discriminator.

The discriminator improves at detecting fakes.

### Step 5 — Repeat for many epochs

The model gradually learns the full distribution.

---

## 5. Why Synthetic Data Is Non-Invertible

---

Synthetic data is **not** a transformation of original rows.

It is **new data** generated from learned distributions.

This means:

- no row corresponds to a real person
- no mapping table exists
- no reconstruction is possible
- no linkage to original individuals exists

Even if the synthetic data resembles the original statistically, it contains **no original values**.

This is fundamentally different from pseudonymization.

---

## 6. How CTGAN Preserves Statistical Properties

---

CTGAN learns:

### A. Marginal Distributions

For example:

- age distribution



- frequency of cities
- name length patterns

## B. Joint Distributions

Relationships between columns:

- older people may appear in certain cities
- certain names may correlate with certain email patterns

## C. Multimodal Patterns

If a column has multiple peaks (e.g., ages 25 and 60), CTGAN preserves them.

## D. Rare Categories

Conditional sampling ensures rare values are not lost.

---

# 7. How CTGAN Avoids Memorizing Real Data

---

GANs are designed to:

- generate new samples
- avoid copying training data

CTGAN includes:

- gradient penalties
- mode-specific sampling
- conditional training
- regularization

These techniques prevent memorization.

---

# 8. How CTGAN Fits Into the Notebook Workflow

---

The notebook uses:

```
synthesizer = CTGANSynthesizer(metadata)
synthesizer.fit(df)
synthetic_df = synthesizer.sample(num_rows=10)
```

This produces:

- a new table
- with the same structure
- with similar statistical properties
- with no real individuals

This is the **non-invertible anonymization** step for structured data.

---

# 9. Interpretation of the Synthetic Table

---

Our synthetic table looked like:

| id       | name                  | email                  | city           | age |
|----------|-----------------------|------------------------|----------------|-----|
| 5451002  | Ahmed Otto            | hayesjorge@example.net | Timothytown    | 78  |
| 6046429  | Irmingard Kitzmann    | yadams@example.net     | Duanehaven     | 29  |
| 11509909 | Dr. Albina Jacobi ... | taylor17@example.net   | Summersborough | 53  |
| ...      |                       |                        |                |     |

Let's analyze this.

---

## 9.1 Some Names Are Reused

CTGAN sometimes reuses names from the original dataset.

This is expected because:

- names are categorical
- CTGAN learns their distribution
- synthetic rows may include the same categories

This is **not** a privacy issue because:

- the rows are new combinations
- the individuals do not exist
- the dataset is synthetic

---

## 9.2 New Cities Are Generated

Cities like:

- "Timothytown"
- "Duanehaven"
- "Summersborough"

are synthetic.

This demonstrates:

- CTGAN's ability to generate new categories
- the model's creativity
- the non-invertible nature of the data

---

## 9.3 New Email Addresses Are Generated

Emails like:

- "hayesjorge@example.net"
- "yadams@example.net"

are synthetic.

They do not correspond to real individuals.

---

## 9.4 Statistical Patterns Are Preserved

The synthetic table:

- has similar age ranges

- has similar name patterns
- has similar email structures
- has similar city distributions

This confirms that CTGAN learned the dataset correctly.

---

## 10. Summary of Chapter 11

---

In this chapter, we learned:

- what SDV and CTGAN are designed to do
- how metadata extraction works
- how CTGAN learns tabular distributions
- how mixed data types are handled
- how conditional sampling preserves rare categories
- why synthetic data is non-invertible
- how CTGAN fits into the anonymization pipeline
- how to interpret the synthetic table

We now understand the architecture behind synthetic data generation.

---

## Chapter 12/15 — Generating Synthetic Tabular Data with SDV: Sampling, Interpretation, and Privacy Guarantees

---

In Chapter 11, we explored the architecture of SDV and the CTGAN model:

how it learns distributions, handles mixed data types, and avoids memorizing real individuals.

This chapter focuses on the **sampling process**—the moment when the trained CTGAN model generates **new synthetic rows**.

This is the point where anonymization becomes **irreversible**.

We will cover:

1. How sampling works in SDV
2. How CTGAN generates new rows
3. Why synthetic data is fundamentally different from pseudonymized data
4. How to interpret the synthetic table
5. How to evaluate privacy guarantees
6. How to evaluate utility and realism
7. Strengths and limitations of synthetic data
8. How synthetic data fits into the overall anonymization pipeline

This chapter is essential because it explains the final output of the notebook:  
a fully synthetic, non-invertible dataset.

---

## 1. The Sampling Step in SDV

---

After training the CTGAN model, the notebook calls:

```
synthetic_df = synthesizer.sample(num_rows=10)
```

This instructs SDV to:

- generate **10 new rows**
- using the learned statistical model
- without copying any original row
- without referencing any original value

This is the core of synthetic data generation.

---

## 2. How CTGAN Generates New Rows

---

CTGAN uses a **generator network** that takes:

- random noise
- conditional vectors
- learned distributions

and produces:

- synthetic latent vectors
- which are decoded into tabular rows

Let's break this down.

---

### 2.1 Random Noise

GANs begin with a random vector  $z$  drawn from a uniform or normal distribution.

This ensures:

- diversity
  - unpredictability
  - non-invertibility
- 

### 2.2 Conditional Vectors

CTGAN uses conditional vectors to:

- ensure rare categories appear
- preserve categorical distributions
- maintain relationships between columns

Example:

If "city" influences "age", CTGAN learns this dependency.

---

### 2.3 Latent Representation

Each column is encoded into a latent space:

- numerical columns → continuous embeddings
- categorical columns → one-hot vectors
- mixed columns → multi-modal encodings

The generator outputs latent vectors that are then decoded.

---

### 2.4 Decoding into Realistic Values

The decoder transforms latent vectors into:

- integers
- strings
- categories
- continuous values

This produces a synthetic row.

---

## 2.5 Repeating the Process

The generator produces one row per iteration.

Sampling 10 rows means:

- 10 independent generations
- each based on random noise
- each statistically consistent with the training data

---

## 3. Why Synthetic Data Is Fundamentally Different from Pseudonymized Data

Synthetic data:

- does **not** contain original values
- does **not** contain hashed values
- does **not** contain masked values
- does **not** contain transformed values
- does **not** correspond to real individuals

Instead, it contains:

- new combinations
- new names
- new emails
- new cities
- new ages

that **never existed** in the original dataset.

This is what makes synthetic data **non-invertible**.

---

## 4. Interpretation of the Synthetic Table

Our synthetic table looked like:

| id       | name                  | email                     | city               | age |
|----------|-----------------------|---------------------------|--------------------|-----|
| 5451002  | Ahmed Otto            | hayesjorge@example.net    | Timothytown        | 78  |
| 6046429  | Irmingard Kitzmann    | yadams@example.net        | Duanehaven         | 29  |
| 11509909 | Dr. Albina Jacobi ... | taylor17@example.net      | Summersborough     | 53  |
| 7602670  | Ahmed Otto            | anthony58@example.com     | West Jenniferhaven | 29  |
| 5066721  | Wilhelmine Döring     | garciaheather@example.com | Guerraland         | 52  |
| ...      |                       |                           |                    |     |

Let's analyze this output in detail.

## 4.1 Synthetic IDs

IDs such as:

- 5451002
- 6046429
- 11509909

are **synthetic**.

CTGAN learned that:

- IDs are integers
- IDs vary widely
- IDs do not repeat

It generates new IDs accordingly.

---

## 4.2 Synthetic Names

Some names appear in the original dataset:

- "Ahmed Otto"
- "Irmgard Kitzmann"
- "Dr. Albina Jacobi ..."

This is expected because:

- names are categorical
- CTGAN learns their distribution
- synthetic rows may reuse categories

This is **not** a privacy issue because:

- the rows are new combinations
  - the individuals do not exist
  - the dataset is synthetic
- 

## 4.3 Synthetic Emails

Emails such as:

- "hayesjorge@example.net"
- "yadams@example.net"
- "taylor17@example.net"

are **synthetic**.

They do not correspond to real individuals.

CTGAN learned:

- email structure
- domain patterns
- username patterns

and generated new values.

---

## 4.4 Synthetic Cities

Cities such as:

- "Timothytown"
- "Duanehaven"
- "Summersborough"

are **completely synthetic**.

This demonstrates:

- CTGAN's ability to generate new categories
- the model's creativity
- the non-invertible nature of the data

---

## 4.5 Synthetic Ages

Ages such as:

- 78
- 29
- 53

fall within the learned distribution.

CTGAN preserves:

- age range
- age distribution shape
- correlations with other columns

---

## 5. Privacy Guarantees of Synthetic Data

Synthetic data provides strong privacy guarantees because:

### A. No row corresponds to a real person

Even if a name appears, the combination of:

- name
- email
- city
- age

is new.

### B. No mapping table exists

Unlike pseudonymization, synthetic data cannot be reversed.

### C. No original values are present

CTGAN generates new values from learned distributions.

### D. No linkage attacks are possible

There is no one-to-one mapping between original and synthetic rows.

### E. No re-identification is possible

Even rare values are transformed into synthetic equivalents.

---

## 6. Utility and Realism of Synthetic Data

---

Synthetic data preserves:

### A. Statistical properties

- distributions
- correlations
- frequencies

### B. Structural properties

- column types
- value ranges
- categorical patterns

### C. Analytical utility

Synthetic data can be used for:

- prototyping
- testing
- analytics
- machine learning

without exposing real individuals.

---

## 7. Strengths of Synthetic Data

---

### A. Strong privacy

No original values remain.

### B. High utility

Statistical patterns are preserved.

### C. Flexible

Can be shared externally.

### D. Safe

No mapping table to protect.

### E. Scalable

Can generate unlimited rows.

---

## 8. Limitations of Synthetic Data

---

### A. Not suitable for all use cases

If exact values are needed, synthetic data is insufficient.

### B. May distort rare categories



CTGAN tries to preserve them, but not perfectly.

### C. Requires training time

GANs are computationally expensive.

### D. Requires careful evaluation

Synthetic data must be validated for:

- privacy
- utility
- distribution fidelity

---

## 9. How Synthetic Data Fits Into the Overall Pipeline

---

The notebook demonstrates two approaches:

### A. Reversible pseudonymization (Chapters 5–6, 9–10)

- deterministic
- invertible
- preserves original values

### B. Irreversible anonymization (Chapters 7, 11–12)

- synthetic data
- no mapping
- no original values

Together, they form a complete anonymization toolkit.

---

## 10. Summary of Chapter 12

---

In this chapter, we learned:

- how SDV samples synthetic rows
- how CTGAN generates new data
- why synthetic data is non-invertible
- how to interpret the synthetic table
- privacy guarantees of synthetic data
- strengths and limitations of the approach
- how synthetic data fits into the anonymization pipeline

We now understand the final output of the notebook:  
a fully synthetic, privacy-preserving dataset.

---

## Chapter 13/15 — Comparing Pseudonymization and Anonymization: Concepts, Trade-offs, and Practical Implications

---

By this point in the notebook, we have seen two fundamentally different approaches to protecting sensitive information:

- **Deterministic, invertible pseudonymization** (text + tables)

- **Non-invertible anonymization** (text placeholders + SDV synthetic data)

This chapter brings these two worlds together and provides a deep comparative analysis. Understanding the differences is essential for choosing the right technique for a given use case.

We will cover:

1. Conceptual differences between pseudonymization and anonymization
2. Technical differences in how each method works
3. Privacy guarantees and risks
4. Utility and analytical value
5. When to use which technique
6. How both approaches complement each other
7. A structured comparison table
8. Interpretation of our notebook outputs in this context

This chapter is one of the most important in the entire series because it synthesizes everything learned so far.

---

# 1. Conceptual Differences

---

## 1.1 Pseudonymization

Pseudonymization replaces sensitive values with **tokens** that:

- are deterministic
- are reversible
- preserve linkability
- preserve uniqueness
- preserve structure

Examples:

- PERSON\_dddfeb9b5b
- NAME\_999cac3df1
- EMAIL\_b99b848bc6

Pseudonymization **protects** data but does not **destroy** it.

---

## 1.2 Anonymization

Anonymization transforms data in a way that:

- cannot be reversed
- removes identifying information
- breaks linkability
- destroys uniqueness
- eliminates re-identification risk

Examples:

- [PERSON]
- [EMAIL]
- synthetic rows generated by SDV

Anonymization **destroys** the original values.

---

# 2. Technical Differences

---

## 2.1 How Pseudonymization Works

- Uses hashing
- Uses deterministic token generation
- Requires mapping tables
- Preserves original structure
- Preserves relationships

Example from our notebook:

```
"Ahmed Otto" → NAME_999cac3df1
```

---

## 2.2 How Anonymization Works

### Text anonymization

- Uses Presidio operators
- Replaces entities with placeholders
- No mapping table
- No reversibility

### Tabular anonymization

- Uses SDV CTGAN
- Generates new rows
- No original values remain
- No mapping table
- No reversibility

---

## 3. Privacy Guarantees

### 3.1 Pseudonymization

Guarantees:

- Direct identifiers are hidden
- Hashes cannot be reversed
- Tokens reveal no semantic information

Risks:

- Mapping table can restore original values
- Frequency patterns remain
- Unique values remain unique
- Linkability remains

Pseudonymization is **not** anonymization.

---

### 3.2 Anonymization

Guarantees:

- No original values remain
- No mapping table exists

- No linkage to real individuals
- No re-identification possible

Risks:

- Synthetic data may distort rare categories
- Utility may be reduced

Anonymization provides **strong privacy**.

---

## 4. Utility and Analytical Value

---

### 4.1 Utility of Pseudonymization

High utility because:

- structure is preserved
- relationships are preserved
- uniqueness is preserved
- linkability is preserved

We can:

- group by pseudonymized names
  - join tables
  - perform analytics
  - debug pipelines
- 

### 4.2 Utility of Anonymization

Moderate to high utility because:

- distributions are preserved
- correlations are preserved
- structure is preserved

But:

- individual-level accuracy is lost
- rare values may be distorted
- synthetic categories may appear

Synthetic data is ideal for:

- prototyping
  - demos
  - external sharing
  - ML training
- 

## 5. When to Use Which Technique

---

### 5.1 We Use Pseudonymization When:

- We need reversibility
- We need to debug pipelines
- We need to link records

- We need to preserve uniqueness
- We need deterministic behavior

Examples:

- internal analytics
- internal testing
- data lineage
- quality checks

---

## 5.2 We Use Anonymization When:

- We need strong privacy
- We need to share data externally
- We need to eliminate re-identification risk
- We need to remove all original values
- We need synthetic data for ML

Examples:

- demos
- public datasets
- research
- training models
- external collaboration

---

# 6. How Both Approaches Complement Each Other

The notebook demonstrates a **hybrid strategy**:

### A. Pseudonymization for internal workflows

- reversible
- consistent
- useful for debugging

### B. Anonymization for external workflows

- irreversible
- privacy-preserving
- safe to share

This hybrid approach is common in modern data engineering.

---

# 7. Structured Comparison Table

| Aspect           | Pseudonymization | Anonymization |
|------------------|------------------|---------------|
| Reversible       | Yes              | No            |
| Mapping table    | Required         | Not used      |
| Linkability      | Preserved        | Destroyed     |
| Uniqueness       | Preserved        | Not preserved |
| Privacy strength | Medium           | High          |

| Aspect                        | Pseudonymization    | Anonymization |
|-------------------------------|---------------------|---------------|
| Utility                       | High                | Medium–High   |
| Suitable for external sharing | No                  | Yes           |
| Suitable for debugging        | Yes                 | No            |
| Contains original values      | No (but reversible) | No            |
| Risk of re-identification     | Medium              | Very low      |

This table summarizes the core differences.

## 8. Interpretation of Our Notebook Outputs in This Context

Let’s revisit our outputs through the lens of this comparison.

### 8.1 Text Pseudonymization Output

```
My name is PERSON_dddfab9b5b ...
```

This is:

- reversible
- deterministic
- linkable
- high utility

### 8.2 Text Anonymization Output

```
My name is [PERSON] ...
```

This is:

- irreversible
- privacy-preserving
- low linkability
- ideal for external sharing

### 8.3 Table Pseudonymization Output

```
NAME_999cac3df1
EMAIL_b99b848bc6
```

This is:

- reversible
- consistent
- ideal for internal analytics

## 8.4 Synthetic Table Output

```
Timothytown  
Duanehaven  
Summersborough
```

This is:

- irreversible
- synthetic
- privacy-preserving
- ideal for demos and ML

---

## 9. Summary of Chapter 13

---

In this chapter, we learned:

- the conceptual differences between pseudonymization and anonymization
- the technical differences in how each method works
- the privacy guarantees and risks of each approach
- the utility and analytical value of each method
- when to use pseudonymization vs. anonymization
- how both approaches complement each other
- how to interpret our notebook outputs in this context

This chapter ties together the reversible and irreversible parts of the pipeline.

---

## Chapter 14/15 — Strengths, Limitations, and Best-Practice Guidance for Presidio + SDV Pipelines

---

By now, we have seen the full workflow:

- PII detection in text
- deterministic pseudonymization (text + tables)
- reversible inversion
- non-invertible anonymization (text placeholders)
- synthetic data generation with SDV

This chapter steps back from the code and focuses on **evaluation**:

What are the strengths of this approach?

What are the limitations?

What best practices should be followed when building real anonymization pipelines?

This chapter is essential because it provides the **practical, architectural, and conceptual insights** needed to use Presidio + SDV effectively and responsibly.

We will cover:

1. Strengths of Presidio for text anonymization
2. Strengths of deterministic pseudonymization
3. Strengths of SDV for synthetic tabular data
4. Limitations of each technique
5. Common pitfalls and how to avoid them
6. Best-practice recommendations

7. How these tools complement each other
8. How to evaluate anonymization quality

This chapter prepares the ground for the final summary in Chapter 15.

---

# 1. Strengths of Presidio for Text Anonymization

---

Presidio is a powerful framework for detecting and transforming PII in unstructured text. Its strengths include:

---

## 1.1 Modular Architecture

Presidio separates:

- **AnalyzerEngine** (PII detection)
- **AnonymizerEngine** (PII transformation)

This modularity allows:

- custom detection logic
- custom anonymization logic
- flexible pipelines

---

## 1.2 High-Quality PII Detection

Presidio uses:

- spaCy NER
- regex-based recognizers
- context-based scoring

This enables detection of:

- names
- emails
- phone numbers
- locations
- IP addresses
- credit cards
- dates
- and more

---

## 1.3 Extensibility

We can add:

- custom recognizers
- custom operators
- custom NLP models

This makes Presidio adaptable to:

- domain-specific identifiers
  - non-English languages
  - specialized formats
-



## 1.4 Deterministic Behavior

Presidio's detection is deterministic when:

- the same text is analyzed
- the same recognizers are used
- the same model version is loaded

This is essential for reproducibility.

---

## 2. Strengths of Deterministic Pseudonymization

---

Deterministic pseudonymization is a powerful technique for internal workflows.

---

### 2.1 Reversible

Mapping tables allow:

- reconstruction
  - debugging
  - audits
  - lineage tracking
- 

### 2.2 Consistent Across Documents

Same input → same token.

This enables:

- linking
  - grouping
  - joins
  - deduplication
- 

### 2.3 Privacy-Preserving

Tokens:

- reveal nothing about the original value
  - are irreversible without the mapping table
  - are safe for internal analytics
- 

### 2.4 Easy to Implement

Hashing + prefixing is:

- simple
  - robust
  - fast
  - language-agnostic
- 

## 3. Strengths of SDV for Synthetic Tabular Data

---

SDV provides a fundamentally different approach: **generate new data instead of transforming existing data.**

---

## 3.1 Strong Privacy Guarantees

Synthetic data:

- contains no original values
- contains no mapping table
- cannot be reversed
- cannot be linked to real individuals

---

## 3.2 High Utility

SDV preserves:

- distributions
- correlations
- categorical frequencies
- numerical ranges

This makes synthetic data useful for:

- analytics
- prototyping
- ML training
- demos

---

## 3.3 Flexibility

SDV supports:

- single-table models
- multi-table relational models
- time-series models
- GAN-based models
- copula-based models

---

## 3.4 Scalability

We can generate:

- 10 rows
- 10,000 rows
- 10 million rows

from the same trained model.

---

# 4. Limitations of Each Technique

No anonymization technique is perfect.

Understanding limitations is essential for responsible use.

---

## 4.1 Limitations of Presidio

### A. Language Dependence

spaCy models are language-specific.

German text requires:

- custom recognizers
- German NLP models

## B. Overlapping Entities

Emails may be split into:

- URL
- DOMAIN
- EMAIL\_ADDRESS

This complicates pseudonymization.

## C. False Positives / False Negatives

PII detection is not perfect.

---

## 4.2 Limitations of Deterministic Pseudonymization

### A. Mapping Table Risk

If leaked, reversibility becomes a privacy issue.

### B. Frequency Attacks

Rare values remain rare.

### C. Not Suitable for External Sharing

Tokens can be guessed or brute-forced.

---

## 4.3 Limitations of SDV Synthetic Data

### A. Training Time

GANs require:

- GPU acceleration for large datasets
- many epochs
- careful tuning

### B. Distortion of Rare Categories

Rare values may be:

- underrepresented
- overrepresented
- smoothed out

### C. Loss of Individual-Level Accuracy

Synthetic data is not suitable when:

- exact values are required
  - precise relationships matter
-

## 5. Common Pitfalls and How to Avoid Them

---

### 5.1 Pitfall: Overlapping Entities in Text

Solution:

- merge overlapping spans
  - filter out URL recognizers
  - prioritize EMAIL\_ADDRESS over URL
- 

### 5.2 Pitfall: Incorrect Language Model

Solution:

- use the correct spaCy model
  - add custom recognizers for non-English text
- 

### 5.3 Pitfall: Mapping Table Mismanagement

Solution:

- encrypt mapping tables
  - store them separately
  - restrict access
- 

### 5.4 Pitfall: Misinterpreting Synthetic Data

Synthetic data is **not**:

- a copy
- a transformation
- a masked version

It is **new data**.

---

## 6. Best-Practice Recommendations

---

### 6.1 For Text Pipelines

- Use entity filtering
  - Merge overlapping entities
  - Use custom recognizers for domain-specific PII
  - Validate detection results manually
- 

### 6.2 For Pseudonymization

- Use SHA-256 or stronger
  - Use prefixes for readability
  - Use separate mappings per column
  - Protect mapping tables with encryption
-

## 6.3 For Synthetic Data

- Validate distributions
  - Validate correlations
  - Validate privacy metrics
  - Use conditional sampling for rare categories
- 

## 6.4 For Hybrid Pipelines

Combine:

- pseudonymization for internal workflows
- synthetic data for external workflows

This provides the best balance of:

- utility
  - privacy
  - flexibility
- 

# 7. How Presidio and SDV Complement Each Other

---

Presidio handles:

- unstructured text
- entity detection
- reversible transformations
- placeholder anonymization

SDV handles:

- structured data
- irreversible anonymization
- synthetic generation

Together, they form a **complete anonymization toolkit**.

---

# 8. Evaluating Anonymization Quality

---

A robust anonymization pipeline should be evaluated along three dimensions:

---

## 8.1 Privacy

- Are original values removed?
  - Are mapping tables protected?
  - Are synthetic rows non-invertible?
- 

## 8.2 Utility

- Are distributions preserved?
  - Are correlations preserved?
  - Are synthetic values realistic?
-

## 8.3 Consistency

- Are pseudonymized values deterministic?
- Are synthetic values stable across runs?

---

## 9. Summary of Chapter 14

---

In this chapter, we learned:

- the strengths of Presidio, pseudonymization, and SDV
- the limitations of each technique
- common pitfalls and how to avoid them
- best-practice recommendations
- how Presidio and SDV complement each other
- how to evaluate anonymization quality

This chapter provides the practical guidance needed to use these tools effectively.

---

## Chapter 15/15 — Final Synthesis, Architectural Insights, and the Future of Modern Anonymization Pipelines

---

This final chapter brings together everything explored across the previous fourteen chapters.

We have examined:

- PII detection in text
- deterministic pseudonymization (text + tables)
- reversible inversion
- non-invertible anonymization
- synthetic data generation with SDV
- strengths, limitations, and best practices

Now we conclude by synthesizing these concepts into a coherent architectural perspective.

This chapter answers the question:

**What does this entire notebook demonstrate, and why does it matter?**

We will cover:

1. The overarching architecture of the pipeline
2. The complementary roles of Presidio and SDV
3. The conceptual evolution from reversible to irreversible anonymization
4. The practical implications for real-world systems
5. The importance of hybrid anonymization strategies
6. The future of privacy-preserving data engineering
7. A final reflection on the notebook's results

This chapter is the capstone of the entire series.

---

## 1. The Overarching Architecture of the Pipeline

---

The notebook demonstrates a **complete anonymization architecture** that spans both unstructured and structured data.

It includes:

## A. Textual PII Detection

Using Presidio's AnalyzerEngine to identify:

- names
- emails
- phone numbers
- locations

## B. Reversible Pseudonymization

Using deterministic hashing to produce stable tokens.

## C. Irreversible Text Anonymization

Using Presidio's AnonymizerEngine to replace PII with placeholders.

## D. Structured Data Pseudonymization

Using deterministic hashing for names and emails in tables.

## E. Synthetic Data Generation

Using SDV's CTGAN to produce new, non-invertible tabular data.

This architecture covers the full spectrum of anonymization needs.

---

# 2. The Complementary Roles of Presidio and SDV

---

Presidio and SDV are not competing tools.

They solve **different problems** and excel in **different domains**.

---

## 2.1 Presidio: Unstructured Text

Presidio is ideal for:

- documents
- logs
- messages
- free-form text

It provides:

- high-quality PII detection
- flexible anonymization operators
- deterministic pseudonymization
- placeholder-based anonymization

Presidio is the backbone of text privacy.

---

## 2.2 SDV: Structured Tabular Data

SDV is ideal for:

- databases
- CSV files
- analytical tables

It provides:

- synthetic data generation
- distribution preservation
- correlation preservation
- strong privacy guarantees

SDV is the backbone of structured data privacy.

---

## 3. The Evolution from Reversible to Irreversible Anonymization

---

The notebook demonstrates a **progression**:

### Step 1 — Detect PII

Identify sensitive values.

### Step 2 — Pseudonymize (Reversible)

Replace values with deterministic tokens.

### Step 3 — Anonymize (Irreversible)

Replace values with placeholders or generate synthetic data.

This progression mirrors real-world anonymization workflows:

- internal teams need reversibility
- external teams need irreversibility
- synthetic data provides the strongest privacy

The notebook shows how to implement all three.

---

## 4. Practical Implications for Real-World Systems

---

The techniques demonstrated in the notebook have direct applications in:

### A. Data Engineering Pipelines

- ETL workflows
- data quality checks
- lineage tracking
- debugging

### B. Machine Learning

- training models on synthetic data
- sharing datasets with external teams
- reducing privacy risk

### C. Analytics

- performing analysis without exposing PII
- preserving statistical properties

### D. Compliance



- GDPR
- CCPA
- internal privacy policies

The notebook's architecture aligns with modern privacy requirements.

---

## 5. The Importance of Hybrid Anonymization Strategies

---

No single anonymization technique is sufficient for all use cases.

The notebook demonstrates a **hybrid strategy**:

### A. Pseudonymization for internal workflows

- reversible
- deterministic
- high utility

### B. Anonymization for external workflows

- irreversible
- privacy-preserving
- safe for sharing

### C. Synthetic data for maximum privacy

- no original values
- no mapping table
- no re-identification risk

This hybrid approach is the gold standard in modern data engineering.

---

## 6. The Future of Privacy-Preserving Data Engineering

---

The techniques demonstrated in this notebook represent the **current state of the art**, but the field is evolving rapidly.

Future developments include:

### A. Differential Privacy Integration

Adding noise to:

- distributions
- queries
- synthetic data

### B. Federated Learning

Training models without centralizing data.

### C. Advanced Generative Models

Using:

- diffusion models
- transformer-based generators
- hybrid GAN architectures

## D. Automated Privacy Validation

Tools that automatically evaluate:

- privacy leakage
- distribution fidelity
- re-identification risk

## E. Domain-Specific PII Detection

Custom recognizers for:

- medical data
- financial data
- legal documents

Presidio and SDV are foundational tools that will integrate into these future systems.

---

# 7. Final Reflection on the Notebook's Results

---

The notebook successfully demonstrates:

## A. Accurate PII Detection

Presidio correctly identifies:

- names
- emails
- phone numbers
- locations

## B. Clean Deterministic Pseudonymization

Tokens are:

- stable
- reversible
- readable
- consistent

## C. Perfect Inversion for Tables

Structured data pseudonymization is fully reversible.

## D. Clean Irreversible Text Anonymization

Placeholders remove all identifying information.

## E. High-Quality Synthetic Data

SDV generates:

- realistic
- statistically consistent
- privacy-preserving

tabular data.

## F. A Complete Anonymization Pipeline

The notebook covers:

- detection
- pseudonymization
- inversion
- anonymization
- synthetic generation

This is a full, end-to-end anonymization architecture.

---

## 8. Summary of Chapter 15

---

In this final chapter, we learned:

- how the entire pipeline fits together
- why Presidio and SDV complement each other
- how reversible and irreversible anonymization differ
- how these techniques apply to real-world systems
- why hybrid anonymization strategies are essential
- how the field is evolving
- what the notebook ultimately demonstrates

This chapter concludes our code analysis.

---



## Final Words

---

We now have a complete, expert-level understanding of:

- PII detection
- pseudonymization
- anonymization
- synthetic data generation
- Presidio
- SDV
- hybrid privacy architectures

This is a comprehensive foundation for building modern, privacy-preserving data systems.